

Protección y Seguridad de Hardware

Metas buscadas

A medida que los sistemas de computación se han vuelto más sofisticados y omnipresentes en sus aplicaciones, también ha crecido la necesidad de proteger su integridad. La protección se concibió originalmente como un complemento de los sistemas operativos multiprogramados, para que los usuarios no fiables pudieran compartir con seguridad un espacio lógico común, como un directorio de archivos, o un espacio físico común, como la memoria. Los conceptos de protección modernos han evolucionado para aumentar la fiabilidad de cualquier sistema complejo que haga uso de recursos compartidos y esté conectado a plataformas de comunicación inseguras como Internet.

La protección consiste en evitar que se haga un uso indebido de los recursos que están dentro del ámbito del sistema operativo. Para ello deben existir mecanismos y políticas que aseguren que los usuarios sólo acceden a sus propios recursos (archivos, zonas de memoria, etc.). Además, es necesario comprobar que los recursos sólo se usan por aquellos usuarios que tienen derechos de acceso a los mismos. Las políticas de protección y seguridad de hardware, software y datos deben incluirse dentro del sistema operativo, pudiendo afectar a uno o varios componentes de este. En cualquier caso, el sistema operativo debe proporcionar medios para implementar la política de protección deseada por el usuario, así como medios de hacer cumplir dicha política.

Resulta necesario proporcionar protección por varias razones. La más obvia es la necesidad de evitar la violación malintencionada de una restricción de acceso por parte de un usuario. Sin embargo, es más importante la necesidad de garantizar que cada proceso de un sistema utilice los recursos del sistema sólo de forma coherente con las políticas establecidas. Este requisito es imprescindible para un sistema fiable.

La protección puede mejorar la fiabilidad detectando los errores latentes en las interfaces entre los subsistemas componentes. La detección temprana de los errores de interfaz puede evitar a menudo la contaminación de un subsistema sano por un subsistema que funciona mal. Además, un recurso no protegido no puede defenderse del uso (o mal uso) de un usuario no autorizado o incompetente. Un sistema orientado a la protección proporciona medios para distinguir entre el uso autorizado y el no autorizado.

El papel de la protección en un sistema informático es proporcionar un mecanismo para la aplicación de las políticas que rigen el uso de los recursos. Estas políticas pueden establecerse de varias maneras. Algunas se fijan en el diseño del sistema, mientras que otras son formuladas por la dirección de un sistema. Otras son definidas por los usuarios individuales para proteger los

recursos que poseen. Por tanto, un sistema de protección debe tener la flexibilidad necesaria para aplicar diversas políticas.

Las políticas de uso de los recursos pueden variar según la aplicación y pueden cambiar con el tiempo. Por estas razones, la protección ya no es una preocupación exclusiva del diseñador de un sistema operativo. El programador de aplicaciones también necesita utilizar mecanismos de protección para proteger los recursos creados y soportados por un subsistema de aplicaciones contra el mal uso. En este módulo, veremos los mecanismos de protección que debe proporcionar el sistema operativo y que los diseñadores de aplicaciones utilizan también al diseñar su propio software de protección.

Debemos tener presente que los mecanismos son distintos de las políticas. Los primeros determinan cómo se hará algo; mientras que los segundos deciden qué se hará. La separación entre política y mecanismo es importante para la flexibilidad. Es probable que las políticas cambien de un lugar a otro o de un momento a otro. En el peor de los casos, cada cambio de política requeriría un cambio en el mecanismo subyacente. El uso de mecanismos generales nos permite evitar esa situación.

Principios de protección

Los requisitos de seguridad son siempre una cuestión importante en las organizaciones. Por ejemplo, la ley obliga a mantener como privados los datos de los clientes de una empresa. Por ello, cuando se demuestra que se han difundido estos datos, se imponen multas a las compañías que violan la ley. Esto se debe generalmente a que el acceso a dichos datos no está controlado o a que las reglas de acceso son tan laxas que todos los compartimentos se encuentran entremezclados. Otro ejemplo interesante es el de la información reservada. Imagine que dos bancos se quieren fusionar. Conocer esta información a tiempo puede permitir hacer grandes negocios o evitar dicha fusión. Para evitar la difusión de la información, se mantiene en compartimentos pequeños que requieren una acreditación de acceso muy grande. La existencia de un sistema seguro pasa porque exista una política de seguridad que defina claramente la seguridad que proporciona el sistema, independientemente de los mecanismos usados para implementarla.

Es interesante resaltar que los requisitos de seguridad varían de unos sistemas a otros, e incluso entre usuarios distintos dentro del sistema. Imagine que una universidad tiene una computadora compartida entre alumnos y profesores, de forma que los alumnos no puedan tener acceso a Internet, pero los profesores sí. El sistema operativo de esta computadora necesita una política de control de acceso a los recursos según el usuario que solicite dichos accesos. Por eso, cuando se instala un sistema operativo en una computadora con restricciones de seguridad, es necesario saber primero si la política de seguridad que ofrece dicho sistema operativo satisface los requisitos de seguridad de la instalación.

Es decir, si es confiable. Ahora bien, un sistema operativo sólo es confiable respecto a una política de seguridad, es decir, a las características de seguridad que se esperen del sistema.

Con frecuencia, se puede utilizar un principio rector a lo largo de un proyecto, como el diseño de un sistema operativo. Seguir este principio simplifica las decisiones de diseño y mantiene el sistema coherente y fácil de entender. Un principio rector clave y probado a lo largo del tiempo para la protección es el principio de privilegio mínimo. Este principio establece que los programas, usuarios, e incluso sistemas, deben tener los privilegios suficientes para realizar sus tareas.

Consideremos uno de los principios de UNIX: un usuario no debe ejecutarse como root. En UNIX, sólo el usuario root puede ejecutar comandos privilegiados. La mayoría de los usuarios respetan esto de forma innata, por temor a una operación de borrado accidental para la que no existe la correspondiente función de anulación. Debido a que root es virtualmente omnipotente, el potencial de error humano cuando un usuario actúa como root es grave, y sus consecuencias son de largo alcance.

Ahora consideremos que más que un error humano, el daño puede ser el resultado de un ataque malintencionado. Un virus lanzado por un clic accidental en un archivo adjunto es un ejemplo. Otro es un desbordamiento del buffer u otro ataque de inyección de código que se lleva a cabo con éxito contra un proceso con privilegios de root o, en Windows, un proceso con privilegios de administrador. Cualquiera de estos casos puede resultar extremadamente dañino para el sistema.

Observar el principio de privilegio mínimo daría al sistema una oportunidad de mitigar el ataque, si el código dañino no puede obtener privilegios de root, existe la posibilidad de que los permisos adecuadamente definidos puedan bloquear todas, o al menos algunas, de las operaciones dañinas. En este sentido, los permisos pueden actuar como un sistema inmunitario a nivel del sistema operativo.

El principio de privilegio mínimo adopta muchas formas, algunas de las cuales veremos más adelante. Otro principio importante, a menudo visto como un derivado del principio de privilegio mínimo, es la compartimentación. La compartimentación es el proceso que consiste en proteger cada componente individual del sistema mediante el uso de permisos específicos y restricciones de acceso. Entonces, si un componente es alterado, actuará otra línea de defensa y evitará que el atacante siga comprometiendo el sistema. La compartimentación se implementa de muchas formas, desde las zonas desmilitarizadas de la red (DMZ) hasta la virtualización.

El uso cuidadoso de las restricciones de acceso puede ayudar a hacer un sistema más seguro y también puede ser beneficioso para producir una pista de

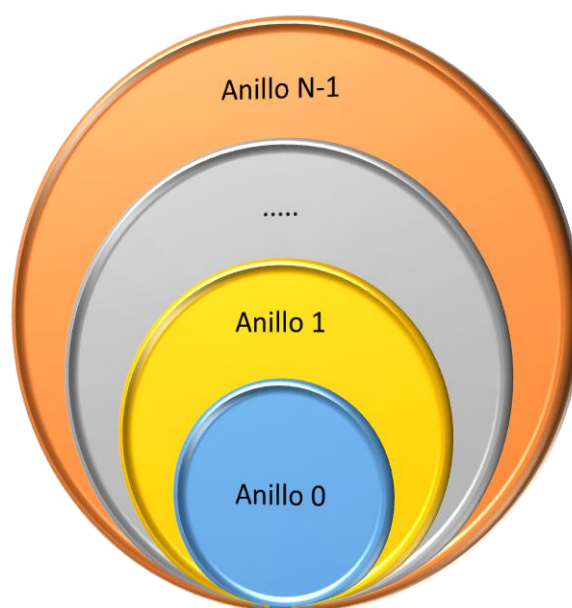
auditoría, que rastrea las divergencias de los accesos permitidos. Una pista de auditoría es un registro duro en los registros del sistema. Si se vigila de cerca, puede revelar alertas tempranas de un ataque o, si su integridad se mantiene a pesar de un ataque, proporcionar pistas sobre qué vectores de ataque se utilizaron, así como evaluar con precisión el daño causado.

Quizás lo más importante es que ningún principio único es la panacea para las vulnerabilidades de seguridad. Debe utilizarse la defensa en profundidad: deben aplicarse múltiples capas de protección, una sobre otra teniendo presente que, obviamente, los atacantes utilizarán múltiples medios para eludir esta defensa en profundidad.

Anillos de protección

Como hemos visto, el componente principal de los sistemas operativos modernos es el núcleo, que gestiona el acceso a los recursos del sistema y al hardware. El núcleo, por definición, es un componente de confianza y privilegiado y, por tanto, debe ejecutarse con un nivel de privilegios superior al de los procesos de usuario.

Para llevar a cabo esta separación de privilegios, es necesario el apoyo del hardware. De hecho, todo el hardware moderno soporta la noción de niveles de ejecución separados, aunque las implementaciones varían un poco. Un modelo popular de separación de privilegios es el de los anillos de protección. En este modelo, creado a partir de Bell-LaPadula, la ejecución se define como un conjunto de anillos concéntricos, en los que el anillo i proporciona un subconjunto de la funcionalidad del anillo j para cualquier $j < i$. El anillo más interno, el anillo 0, proporciona así el conjunto completo de privilegios. Este concepto se muestra en el gráfico siguiente.



Cuando el sistema arranca, lo hace en el nivel de privilegio más alto. El código en ese nivel realiza la inicialización necesaria antes de bajar a un nivel menos privilegiado. Para volver a un nivel de privilegio superior, el código suele llamar a una instrucción especial, a veces denominada puerta, que proporciona un portal entre anillos. La instrucción de llamada al sistema es un ejemplo. La ejecución de esta instrucción cambia la ejecución del modo usuario al modo kernel. Como hemos visto, la ejecución de una llamada al sistema siempre transferirá la ejecución a una dirección predefinida, permitiendo al que llama especificar sólo los argumentos (incluyendo el número de llamada al sistema), y no direcciones arbitrarias del núcleo. De esta manera, la integridad del anillo más privilegiado puede ser generalmente asegurada.

Otra forma de acabar en un anillo más privilegiado es cuando se produce una trampa del procesador o una interrupción. Cuando se produce cualquiera de ellas, la ejecución se transfiere inmediatamente al anillo de mayor privilegio. Una vez más, sin embargo, la ejecución en el anillo de mayor privilegio está predefinida y restringida a una ruta de código bien protegida.

Las arquitecturas de Intel siguen este modelo, colocando el código en modo usuario en el anillo 3 y el código en modo núcleo (Kernel) en el anillo 0. La distinción se hace mediante dos bits en el registro especial EFLAGS. El acceso a este registro no está permitido en el anillo 3, impidiendo así que un proceso malintencionado escale privilegios. Con la llegada de la virtualización, Intel definió un anillo adicional (-1) para permitir los hipervisores, o gestores de máquinas virtuales, que crean y ejecutan máquinas virtuales. Los hipervisores tienen más capacidades que los núcleos de los sistemas operativos invitados.

Inicialmente, la arquitectura del procesador ARM sólo permitía los modos USR y SVC, para el modo usuario y el núcleo (supervisor), respectivamente. En los procesadores ARMv7, ARM introdujo la Zona de Confianza (TZ: TrustZone), que proporcionaba un anillo adicional. Este entorno de ejecución más privilegiado también tiene acceso exclusivo a funciones criptográficas respaldadas por hardware, como el elemento seguro NFC y una clave criptográfica en el chip, que hacen más seguro el manejo de contraseñas e información sensible. Incluso el propio kernel no tiene acceso a la clave en el chip, y sólo puede solicitar servicios de cifrado y descifrado del entorno de la Zona de Confianza, (mediante una instrucción especializada, Secure Monitor Call (SMC)), que sólo es utilizable desde el modo kernel. Al igual que con las llamadas al sistema, el kernel no tiene capacidad para ejecutar directamente a direcciones específicas en la Zona de Confianza, sólo para pasar argumentos a través de registros. Android utiliza ampliamente la Zona de Confianza a partir de la versión 5.0.

Emplear correctamente un entorno de ejecución de confianza significa que, si el kernel se ve comprometido, un atacante no puede simplemente recuperar la clave de la memoria del kernel. Mover los servicios criptográficos a un entorno separado y de confianza también hace que los ataques de fuerza bruta tengan

menos posibilidades de éxito. Estos ataques consisten en probar todas las combinaciones posibles de caracteres válidos para la contraseña hasta encontrarla. Las distintas claves utilizadas por el sistema, desde la contraseña del usuario hasta la propia del sistema, se almacenan en la clave en el chip, a la que sólo se puede acceder en un contexto de confianza. Cuando se introduce una clave, por ejemplo, una contraseña, se verifica mediante una petición al entorno de la zona de confianza. Si no se conoce una clave y hay que adivinarla, el verificador de la zona de confianza puede imponer limitaciones, por ejemplo, limitando el número de intentos de verificación.

En la arquitectura ARMv8 de 64 bits, ARM amplió su modelo para admitir cuatro niveles, llamados niveles de excepción, numerados del EL0 al EL3. El modo de usuario se ejecuta en EL0, y el modo kernel en EL1. El EL2 está reservado para los hipervisores, y el EL3, el más privilegiado, está reservado para el monitor seguro, la capa TrustZone. Cualquiera de los niveles de excepción permite ejecutar sistemas operativos separados uno al lado del otro.

Tenga en cuenta que el monitor seguro se ejecuta en un nivel de ejecución superior al de los kernels de propósito general, lo que lo convierte en el lugar perfecto para desplegar código que compruebe la integridad de los kernels. Esta funcionalidad se incluye en el Realtime Kernel Protection (RKP) de Samsung para Android y en el WatchTower (también conocido como KPP, por Kernel Patch Protection) de Apple para iOS.

Dominios de protección

La seguridad es más fácil de conseguir si existe un modelo claro de lo que debe protegerse y de quién puede hacer qué. Se ha trabajado bastante en este ámbito, por lo que en este breve desarrollo sólo podemos arañar la superficie. Nos centraremos en algunos modelos generales y en los mecanismos para aplicarlos.

Los anillos de protección separan las funciones en dominios y las ordenan jerárquicamente. Una generalización de los anillos es el uso de dominios sin jerarquía. Un sistema informático puede tratarse como una colección de procesos y objetos. Por objetos, nos referimos tanto a los objetos de hardware (como la CPU, los segmentos de memoria, las páginas de memoria, las impresoras, los discos, las unidades de cinta, etc.) como a los objetos de software (como los archivos, los programas, las bases de datos y los semáforos). Cada objeto tiene un nombre único por el que se le hace referencia, es decir que lo diferencia de todos los demás objetos del sistema, y un conjunto finito de operaciones que los procesos pueden realizar sobre él y sólo se puede acceder a cada uno de ellos mediante operaciones bien definidas y significativas. Los objetos son esencialmente tipos de datos abstractos.

Las operaciones posibles dependen del objeto. Por ejemplo, en una CPU sólo podemos ejecutar. Las palabras de memoria pueden leerse y escribirse, mientras que un DVD-ROM sólo puede leerse. Las unidades de cinta pueden leerse, escribirse y rebobinarse. Las operaciones de lectura y escritura son propias de un archivo; las de subida y bajada tienen sentido en un semáforo. Los archivos de datos pueden crearse, abrirse, leerse, escribirse, cerrarse y borrarse; los archivos de programa pueden leerse, escribirse, ejecutarse y borrarse. Es obvio que se necesita una forma de prohibir a los procesos el acceso a objetos a los que no están autorizados a acceder. Además, este mecanismo también debe permitir restringir los procesos a un subconjunto de las operaciones legales cuando sea necesario. Por ejemplo, el proceso A puede estar autorizado a leer, pero no a escribir, el archivo F.

Un proceso debe poder acceder sólo a los objetos para los que tiene autorización. Además, en cualquier momento, un proceso debe poder acceder sólo a los objetos que necesita en ese momento para completar su tarea. Este segundo requisito, el principio de necesidad de conocer, es útil para limitar la cantidad de daño que un proceso defectuoso o un atacante puede causar en el sistema. Por ejemplo, cuando el proceso P invoca al procedimiento A(), el procedimiento debería poder acceder sólo a sus propias variables y a los parámetros formales que se le pasan; no debería poder acceder a todas las variables del proceso P. De forma similar, consideremos el caso en el que el proceso P invoca a un compilador para compilar un archivo concreto. El compilador no debería poder acceder a los archivos de forma arbitraria, sino que debería tener acceso sólo a un subconjunto bien definido de archivos (como el archivo fuente, el archivo objeto de salida y otros) relacionados con el archivo a compilar. Por el contrario, el compilador puede tener archivos privados utilizados con fines de contabilidad u optimización a los que el proceso P no debería poder acceder.

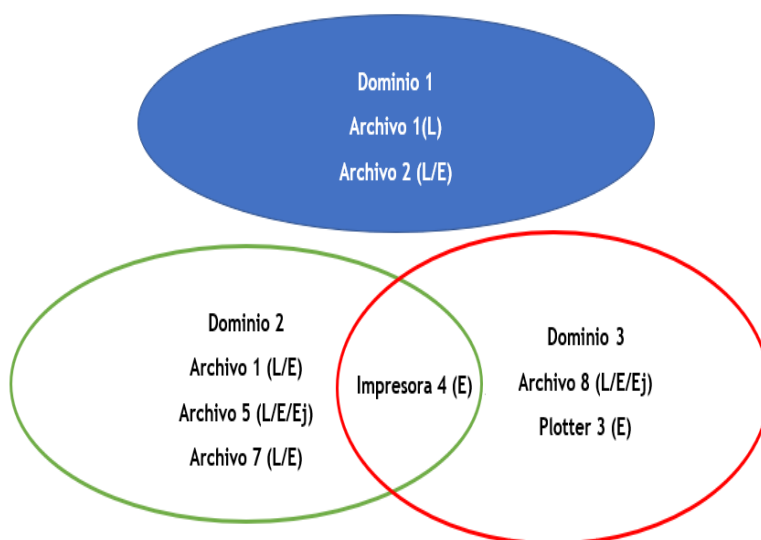
Al comparar la necesidad de conocer con el privilegio mínimo, puede ser más fácil pensar en la necesidad de conocer a la política y al privilegio mínimo como al mecanismo para lograr esta política. Por ejemplo, en los permisos de archivos, la necesidad de conocer puede dictar que un usuario tenga acceso de lectura, pero no de escritura o ejecución de un archivo. El principio de privilegio mínimo requeriría que el sistema operativo proporcionara un mecanismo para permitir el acceso de lectura, pero no el de escritura o de ejecución.

Para discutir los diferentes mecanismos de protección, es útil introducir el concepto de dominio. Un dominio es un conjunto de pares (objeto, derechos). Cada par especifica un objeto y un subconjunto de operaciones que pueden realizarse sobre él. Un derecho en este contexto significa el permiso para realizar una de las operaciones. A menudo, un dominio corresponde a un solo usuario, indicando lo que éste puede hacer y no hacer, pero un dominio también

puede ser más general que un solo usuario. Por ejemplo, los miembros de un equipo de programación que trabajan en algún proyecto pueden pertenecer todos al mismo dominio para que todos puedan acceder a los archivos del proyecto.

La forma en que se asignan los objetos a los dominios depende de la especificidad de quién necesita saber qué. Sin embargo, un concepto básico es el POLA (Principio de autoridad mínima) o necesidad de saber. En general, la seguridad funciona mejor cuando cada dominio tiene los objetos y privilegios mínimos para hacer su trabajo, y no más.

Los dominios pueden compartir los derechos de acceso. La siguiente figura muestra tres dominios, mostrando los objetos en cada dominio y los derechos (Lectura, Escritura, Ejecutar) disponibles en cada objeto. Observemos que Impresora 1 está en dos dominios al mismo tiempo, con los mismos derechos en cada uno y Archivo 1 también está en dos dominios, pero con diferentes derechos en cada uno.



En cada instante de tiempo, cada proceso se ejecuta en algún dominio de protección. En otras palabras, hay una colección de objetos a los que puede acceder, y para cada objeto tiene un conjunto de derechos. Los procesos también pueden pasar de un dominio a otro durante la ejecución. Las reglas para el cambio de dominio dependen en gran medida del sistema.

Si la asociación entre los procesos y los dominios es fija, y queremos adherirnos al principio de necesidad de conocer, entonces debe haber un mecanismo disponible para cambiar el contenido de un dominio. La razón se debe a que un proceso puede ejecutarse en dos fases diferentes y puede, por ejemplo, necesitar acceso de lectura en una fase y de escritura en otra. Si un dominio es estático, debemos definir el dominio para incluir tanto el acceso de lectura como el de escritura. Sin embargo, esta disposición proporciona más derechos de los que se necesitan en cada una de las dos fases, ya que tenemos acceso de

lectura en la fase en la que sólo necesitamos acceso de escritura, y viceversa. Así, se viola el principio de necesidad de conocer. Debemos permitir que se modifique el contenido de un dominio para que éste refleje siempre los derechos de acceso mínimos necesarios.

Si la asociación es dinámica, se dispone de un mecanismo para permitir el cambio de dominio, permitiendo que el proceso cambie de un dominio a otro. También podemos querer permitir que se modifique el contenido de un dominio. Si no podemos cambiar el contenido de un dominio, podemos proporcionar el mismo efecto creando un nuevo dominio con el contenido cambiado y cambiando a ese nuevo dominio cuando queramos cambiar el contenido del dominio.

Un dominio puede constituirse de varias maneras:

- **Cada usuario puede ser un dominio.** En este caso, el conjunto de objetos a los que se puede acceder depende de la identidad del usuario. El cambio de dominio se produce cuando se cambia de usuario, generalmente cuando un usuario se desconecta y otro se conecta.
- **Cada proceso puede ser un dominio.** En este caso, el conjunto de objetos a los que se puede acceder depende de la identidad del proceso. El cambio de dominio se produce cuando un proceso envía un mensaje a otro proceso y luego espera una respuesta.
- **Cada procedimiento puede ser un dominio.** En este caso, el conjunto de objetos a los que se puede acceder corresponde a las variables locales definidas dentro del procedimiento. La conmutación de dominios se produce cuando se realiza una llamada a un procedimiento.

Discutiremos el cambio de dominio con más detalle, pero, mientras tanto, consideremos el modelo estándar de modo dual (kernel - modo usuario) de ejecución del sistema operativo. Cuando un proceso está en modo kernel, puede ejecutar instrucciones privilegiadas y, por lo tanto, obtener el control completo del sistema computador. En cambio, cuando un proceso se ejecuta en modo usuario, sólo puede invocar instrucciones no privilegiadas. En consecuencia, sólo puede ejecutarse dentro de su espacio de memoria predefinido. Estos dos modos protegen al sistema operativo (que se ejecuta en el dominio del kernel) de los procesos de usuario (que se ejecutan en el dominio de usuario). En un sistema operativo multiprogramado, dos dominios de protección son insuficientes, ya que los usuarios también quieren estar protegidos unos de otros. Por lo tanto, para concretar la idea de un dominio de protección, se necesita un esquema más elaborado. Ilustraremos dicho esquema examinando dos sistemas operativos destacados, UNIX y Android, para ver cómo implementan estos conceptos.

En UNIX, el dominio de un proceso está definido por su UID y GID. Cuando un usuario se conecta, su shell obtiene el UID y el GID contenidos en su entrada en

el archivo de contraseñas, y éstos son heredados por todos sus hijos. Dada cualquier combinación (UID, GID), es posible hacer una lista completa de todos los objetos (archivos, incluidos los dispositivos de E/S representados por archivos especiales, etc.) a los que se puede acceder, y si se puede acceder para leer, escribir o ejecutar. Dos procesos con la misma combinación (UID, GID) tendrán acceso a exactamente el mismo conjunto de objetos. Los procesos con diferentes valores (UID, GID) tendrán acceso a un conjunto diferente de archivos, aunque puede haber un solapamiento considerable.

Además, cada proceso en UNIX tiene dos mitades: la parte de usuario y la parte del kernel. Cuando el proceso hace una llamada al sistema, pasa de la parte de usuario a la parte del kernel. La parte del kernel tiene acceso a un conjunto de objetos diferente al de la parte de usuario. Por ejemplo, el kernel puede acceder a todas las páginas de la memoria física, a todo el disco y a todos los demás recursos protegidos. Así, una llamada al sistema provoca un cambio de dominio.

Cuando un proceso realiza una ejecución con un archivo con el bit SETUID o SETGID activado, adquiere un nuevo UID o GID efectivo. Con una combinación diferente (UID, GID), tiene un conjunto diferente de archivos y operaciones disponibles. Ejecutar un programa con SETUID o SETGID es también un cambio de dominio, ya que los derechos disponibles cambian.

En Android, se proporcionan ID's de usuario distintos para cada aplicación. Cuando se instala una aplicación, el demonio instalado le asigna un ID de usuario (UID) y un ID de grupo (GID) distintos, junto con un directorio de datos privado (/data/data/<nombre de la aplicación>) cuya propiedad se concede únicamente a esta combinación de UID/GID. De este modo, las aplicaciones del dispositivo disfrutan del mismo nivel de protección que ofrecen los sistemas UNIX a los usuarios por separado. Esta es una forma rápida y sencilla de proporcionar aislamiento, seguridad y privacidad. El mecanismo se amplía modificando el kernel para permitir ciertas operaciones, como los sockets de red, sólo a los miembros de un GID concreto (por ejemplo, AID_INET, 3003). Otra mejora de Android consiste en definir ciertos UID como aislados, lo que les impide iniciar peticiones RPC a cualquier servicio, excepto a un mínimo.

Matriz de acceso

Una cuestión importante es cómo el sistema mantiene la pista de qué objeto pertenece a qué dominio. Conceptualmente, al menos, se puede imaginar una gran matriz, en la que las filas son los dominios y las columnas los objetos. En cada casilla se enumeran los derechos, si los hay, que el dominio contiene para el objeto. La representación matricial de los diagramas de la figura anterior se muestra en la siguiente tabla T1. Dada esta matriz y el número de dominio actual, el sistema puede decir si se permite un acceso a un objeto dado de una manera particular desde un dominio específico.

T1		Objeto							
		Archivo 1	Archivo 2	Archivo 3	Archivo 5	Archivo 7	Archivo 8	Impresora 4	Plotter 3
Dominio	1	Lectura	Lectura / Escritura						
	2			Lectura	Lectura / Escritura / Ejecución	Lectura / Escritura		Imprimir	
	3						Lectura / Escritura / Ejecución	Imprimir	Imprimir

El cambio de dominio en sí mismo puede incluirse fácilmente en el matricial al darse cuenta de que un dominio es en sí mismo un objeto, con la operación Enter. La siguiente tabla T2 muestra nuevamente a la tabla anterior, sólo que ahora con los tres dominios como objetos en sí mismos. Los procesos en el dominio 1 pueden cambiar al dominio 2, pero una vez allí, no pueden regresar. Esta situación modela la ejecución de un programa SETUID en UNIX. No se permite ningún otro cambio de dominio en este ejemplo.

T2		Objeto										
		Archivo 1	Archivo 2	Archivo 3	Archivo 5	Archivo 7	Archivo 8	Impresora 4	Plotter 3	Dominio 1	Dominio 2	Dominio 3
Dominio	1	Lectura	Lectura / Escritura								Enter	
	2			Lectura	Lectura / Escritura / Ejecución	Lectura / Escritura		Imprimir				
	3						Lectura / Escritura / Ejecución	Imprimir	Imprimir			

El modelo general de protección puede verse de forma abstracta como una matriz, llamada matriz de acceso. Las filas de la matriz de acceso representan dominios, y las columnas representan objetos como podemos apreciar en la primera de las dos tablas anteriores. Cada entrada de la matriz consiste en un conjunto de derechos de acceso. Como la columna define explícitamente los objetos, podemos omitir el nombre del objeto en el derecho de acceso. La entrada Acceso(i,j) define el conjunto de operaciones que un proceso, que se ejecuta en el dominio Di, puede invocar sobre el objeto Oj. La siguiente tabla T3 muestra este concepto.

El esquema de la matriz de acceso nos proporciona el mecanismo para especificar una variedad de políticas. El mecanismo consiste en implementar la matriz de acceso y asegurar que las propiedades semánticas que hemos

esbozado se mantienen. Más concretamente, debemos garantizar que un proceso que se ejecute en el dominio D_i sólo pueda acceder a los objetos especificados en la fila i , y sólo según lo permitido por las entradas de la matriz de acceso. La matriz de acceso puede aplicar decisiones políticas relativas a la protección. Las decisiones políticas implican qué derechos deben incluirse en la entrada (i, j) .

T3		Objeto			
		Archivo 1	Archivo 2	Archivo 3	Impresora
Dominio	1	Lectura		Lectura	
	2				Imprimir
	3		Lectura	Ejecución	
	4	Lectura / Escritura		Lectura / Escritura	

También debemos decidir el dominio en el que se ejecuta cada proceso. Esta última política la suele decidir el sistema operativo. Los usuarios suelen decidir el contenido de las entradas de la matriz de acceso. Cuando un usuario crea un nuevo objeto O_j , la columna O_j se añade a la matriz de acceso con las entradas de inicialización adecuadas, según lo dictado por el creador. El usuario puede decidir introducir algunos derechos en algunas entradas de la columna j y otros derechos en otras entradas, según sea necesario.

La matriz de acceso proporciona un mecanismo apropiado para definir e implementar un control estricto para la asociación estática y dinámica entre procesos y dominios. Cuando cambiamos un proceso de un dominio a otro, estamos ejecutando una operación (switch) sobre un objeto (el dominio). Podemos controlar el cambio de dominio incluyendo los dominios entre los objetos de la matriz de acceso. Del mismo modo, cuando cambiamos el contenido de la matriz de acceso, estamos realizando una operación sobre un objeto: la matriz de acceso. De nuevo, podemos controlar estos cambios incluyendo la propia matriz de acceso como un objeto. En realidad, como cada entrada de la matriz de acceso puede ser modificada individualmente, debemos considerar cada entrada de la matriz de acceso como un objeto a proteger. Ahora, tenemos que considerar sólo las operaciones posibles sobre estos nuevos objetos (dominios y la matriz de acceso) y decidir cómo queremos que los procesos puedan ejecutar estas operaciones.

Los procesos deben poder cambiar de un dominio a otro. El cambio del dominio D_i al dominio D_j está permitido si y sólo si el derecho de acceso switch

pertenece a Acceso(i, j). Así, en la tabla T4, un proceso que se ejecuta en el dominio D2 puede cambiar (Switch) al dominio D3 o al dominio D4. Un proceso en el dominio D4 puede cambiar a D1, y uno en el dominio D1 puede cambiar a D2.

T4		Objeto							
		Archivo 1	Archivo 2	Archivo 3	Impresora	Dominio D1	Dominio D2	Dominio D3	Dominio D4
Dominio	1	Lectura		Lectura			Switch		
	2				Imprimir			Switch	Switch
	3		Lectura	Ejecución					
	4	Lectura / Escritura		Lectura / Escritura		Switch			

Permitir un cambio controlado en el contenido de las entradas de la matriz de acceso requiere tres operaciones adicionales: copia, propietario y control. La capacidad de copiar un derecho de acceso de un dominio (o fila) de la matriz de acceso a otro se denota con un asterisco (*) añadido al derecho de acceso.

El derecho de copia permite que el derecho de acceso se copie sólo dentro de la columna (es decir, para el objeto) para el que se define el derecho. Por ejemplo, en la tabla T5, un proceso que se ejecuta en el dominio D2 puede copiar la operación de lectura en cualquier entrada asociada al archivo F2.

T5		Objeto		
		Archivo F1	Archivo F2	Archivo F3
Dominio	1	Ejecución		Escribir*
	2	Ejecución	Lectura*	Ejecución
	3	Ejecución		

Por lo tanto, la matriz de acceso de la tabla T5 puede ser modificada a la matriz de acceso mostrada en la tabla T6.

T6		Objeto		
		Archivo F1	Archivo F2	Archivo F3
Dominio	1	Ejecución		Escribir*
	2	Ejecución	Lectura*	Ejecución
	3	Ejecución	Lectura	

Este esquema tiene dos variantes adicionales:

- El derecho se copia del Acceso(i, j) al Acceso(k, j); luego se elimina del Acceso(i, j). Esta acción es una transferencia de un derecho, más que una copia.
- La propagación del derecho de copia puede ser limitada. Es decir, cuando el derecho R* se copia del Acceso(i, j) al Acceso(k, j), sólo se crea el derecho R (no R*). Un proceso que se ejecuta en el dominio Dk no puede seguir copiando el derecho R.

Un sistema puede seleccionar sólo uno de estos tres derechos de copia, o puede proporcionar los tres identificándolos como derechos separados: copia, transferencia y copia limitada.

También necesitamos un mecanismo que permita añadir nuevos derechos y eliminar algunos. El derecho de propietario controla estas operaciones. Si el Acceso(i, j) incluye el derecho de propietario, entonces un proceso que se ejecuta en el dominio Di puede añadir y eliminar cualquier derecho en cualquier entrada de la columna j. Por ejemplo, en la tabla T7, el dominio D1 es el propietario de F1 y por tanto puede añadir y eliminar cualquier derecho válido en la columna F1. Del mismo modo, el dominio D2 es el propietario de F2 y F3 y, por tanto, puede añadir y eliminar cualquier derecho válido dentro de estas dos columnas.

T7		Objeto		
		Archivo F1	Archivo F2	Archivo F3
Dominio	1	Propietario Ejecución		Escribir
	2		Lectura* Propietario	Lectura* Propietario Escribir
	3	Ejecución		

Así, la matriz de acceso de la tabla T7 puede modificarse a la matriz de acceso mostrada en la tabla T8.

T8		Objeto		
		Archivo F1	Archivo F2	Archivo F3
Dominio	1	Propietario Ejecución		Escribir
	2		Propietario Lectura* Escribir*	Lectura* Propietario Escribir
	3		Escribir	Escribir

Los derechos de copia y de propietario permiten que un proceso cambie las entradas de una columna. También se necesita un mecanismo para cambiar las entradas de una fila. El derecho de control sólo es aplicable a los objetos del dominio. Si el Acceso(i, j) incluye el derecho de control, entonces un proceso que se ejecuta en el dominio D_i puede eliminar cualquier derecho de acceso de la fila j . Por ejemplo, supongamos que, en la tabla T4, incluimos el derecho de control en el Acceso(D_2, D_4). Entonces, un proceso que se ejecute en el dominio D_2 podría modificar el dominio D_4 , como se muestra en la tabla T9.

T9		Objeto							
		Archivo 1	Archivo 2	Archivo 3	Impresora	Dominio D1	Dominio D2	Dominio D3	Dominio D4
Dominio	1	Lectura		Lectura			Switch		
	2				Impresión			Switch	Switch Control
	3		Lectura	Ejecución					
	4	Escritura		Escritura		Switch			

Los derechos de copia y de propietario nos proporcionan un mecanismo para limitar la propagación de los derechos de acceso. Sin embargo, no nos dan las herramientas adecuadas para evitar la propagación (o divulgación) de la información. El problema de garantizar que ninguna información contenida inicialmente en un objeto pueda migrar fuera de su entorno de ejecución se denomina problema de confinamiento. Este problema es en general irresoluble.

Estas operaciones sobre los dominios y la matriz de acceso no son importantes en sí mismas, pero ilustran la capacidad del modelo de matriz de acceso para permitirnos implementar y controlar los requisitos de protección dinámica. Se pueden crear nuevos objetos y dominios de forma dinámica e incluirlos en el modelo de matriz de acceso. Sin embargo, sólo hemos mostrado que el mecanismo básico existe. Los diseñadores del sistema operativo y los usuarios deben tomar las decisiones políticas sobre qué dominios deben tener acceso a qué objetos y de qué manera.

Implementaciones de la matriz de acceso

¿Cómo se puede implementar eficazmente la matriz de acceso? En general, la matriz será dispersa, es decir, la mayoría de las entradas estarán vacías. Aunque existen técnicas de estructuras de datos para representar matrices dispersas, no son especialmente útiles para esta aplicación, debido a la forma

en que se utiliza la facilidad de protección. Seguidamente, describiremos varias implementaciones de la matriz de acceso.

Tabla global.

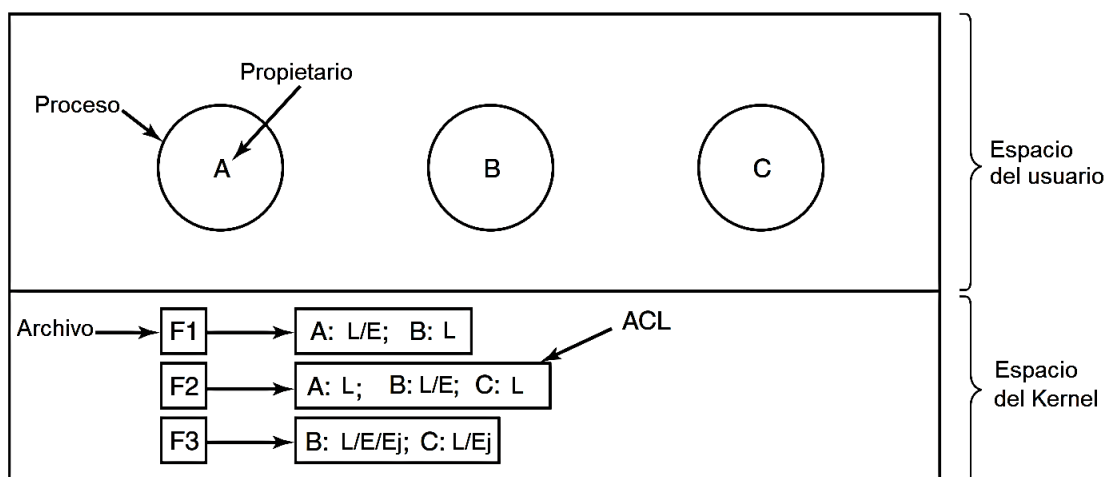
La implementación más sencilla de la matriz de acceso es una tabla global que consiste en un conjunto de ternas ordenadas <dominio, objeto, conjunto de derechos>. Cada vez que se ejecuta una operación M sobre un objeto Oj dentro del dominio Dj, se busca en la tabla global una tripleta <Di, Oj, Rk>, con M perteneciendo a Rk. Si se encuentra esta terna, se permite que la operación continúe; en caso contrario, se lanza una condición de excepción (o error).

Esta implementación tiene varios inconvenientes. La tabla suele ser grande y, por lo tanto, no puede mantenerse en la memoria principal, por lo que se necesita una E/S adicional. A menudo se utilizan técnicas de memoria virtual para gestionar esta tabla. Además, es difícil aprovechar las agrupaciones especiales de objetos o dominios. Por ejemplo, si todo el mundo puede leer un objeto en particular, este objeto debe tener una entrada separada en cada dominio.

Listas de control de acceso a los objetos

En la práctica, rara vez se almacena la matriz de la tabla T2 porque es grande y escasa. La mayoría de los dominios no tienen ningún acceso a la mayoría de los objetos, por lo que almacenar una matriz muy grande, mayormente vacía, es un desperdicio de espacio en el disco. Sin embargo, hay dos métodos prácticos que consisten en almacenar la matriz por filas o por columnas, y luego almacenar sólo los elementos no vacíos. Los dos enfoques son sorprendentemente diferentes. Veremos primero el almacenamiento por columnas y luego el almacenamiento por filas.

La primera técnica consiste en asociar a cada objeto una lista (ordenada) que contiene todos los dominios que pueden acceder al objeto, y cómo. Esta lista se llama ACL (Access Control List) y se ilustra en la siguiente figura.



En ella vemos tres procesos, cada uno perteneciente a un dominio diferente, A, B y C, y tres archivos F1, F2 y F3. Para simplificar, asumiremos que cada dominio corresponde exactamente a un usuario, en este caso, los usuarios A, B y C. Muy frecuentemente en los textos sobre seguridad los usuarios son llamados sujetos o principales, para contrastarlos con las cosas que poseen, los objetos, como los archivos.

Cada archivo tiene una ACL asociada. El archivo F1 tiene dos entradas en su ACL (separadas por un punto y coma). La primera entrada dice que cualquier proceso propiedad del usuario A puede leer y escribir en el archivo. La segunda entrada dice que cualquier proceso propiedad del usuario B puede leer el archivo. Todos los demás accesos de estos usuarios y todos los accesos de otros usuarios están prohibidos. Notemos que los derechos se conceden por usuario, no por proceso. En cuanto al sistema de protección, cualquier proceso propiedad del usuario A puede leer y escribir el archivo F1. No importa si hay un proceso de este tipo o 100 de ellos. Lo que importa es el propietario, no el ID del proceso.

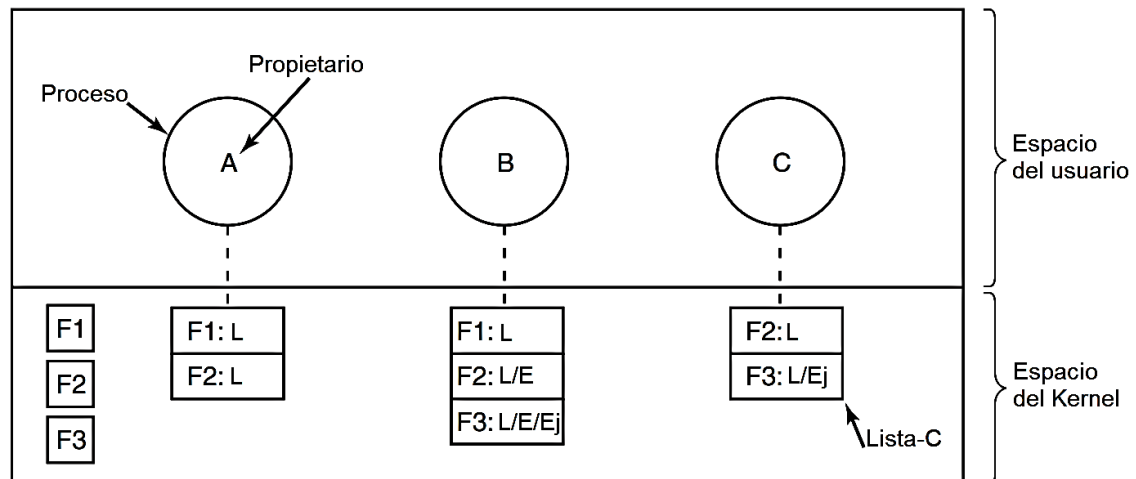
El archivo F2 tiene tres entradas en su ACL: A, B y C pueden leer el archivo, y B también puede escribirlo. No se permite ningún otro acceso. El archivo F3 es aparentemente un programa ejecutable, ya que B y C pueden leerlo y ejecutarlo. B también puede escribirlo. Este ejemplo ilustra la forma más básica de protección con ACL's. En la práctica se suelen utilizar sistemas más sofisticados.

Para empezar, hasta ahora sólo hemos mostrado tres derechos: lectura, escritura y ejecución. También puede haber derechos adicionales. Algunos de ellos pueden ser genéricos, es decir, aplicarse a todos los objetos, y otros pueden ser específicos de un objeto. Ejemplos de derechos genéricos son destruir objeto y copiar objeto. Estos pueden aplicarse a cualquier objeto, sin importar su tipo. Los derechos específicos de un objeto pueden incluir añadir un mensaje para un objeto de buzón y ordenar alfabéticamente un objeto de directorio. Además, hasta ahora, nuestras entradas ACL han sido para usuarios individuales, pero, conviene mencionar que, muchos sistemas soportan el concepto de grupo de usuarios.

Listas de capacidades para los dominios.

La otra forma de dividir la matriz de la tabla T2 es por filas. Cuando se utiliza este método, se asocia a cada proceso una lista de objetos a los que se puede acceder, junto con una indicación de qué operaciones están permitidas en cada uno, en otras palabras, su dominio. Esta lista se denomina lista de capacidades, o lista C, y sus elementos individuales se denominan capacidades. En la siguiente figura mostramos un conjunto de tres procesos y sus listas de capacidades. Cada capacidad concede al propietario ciertos derechos sobre un determinado objeto. En la figura, el proceso propiedad del usuario A puede leer

los archivos F1 y F2, por ejemplo. Normalmente, una capacidad consiste en un identificador de archivo (o más generalmente, de objeto) y un mapa de bits para los distintos derechos. En un sistema tipo UNIX, el identificador de archivo sería probablemente el número de nodo *i*. Las listas de capacidades son en sí mismas objetos y pueden ser apuntadas desde otras listas de capacidades, facilitando así la compartición de subdominios.



Es bastante obvio que las listas de capacidades deben estar protegidas de la manipulación de los usuarios. Se conocen tres métodos para protegerlas. El primero requiere una arquitectura etiquetada, un diseño de hardware en el que cada palabra de memoria tiene un bit extra (o etiqueta) que indica si la palabra contiene una capacidad o no. El bit de etiqueta no es utilizado por las instrucciones aritméticas, de comparación o similares, y sólo puede ser modificado por los programas que se ejecutan en modo kernel, es decir, el sistema operativo. Se han construido máquinas con arquitectura etiquetada y se puede hacer que funcionen bien. El IBM AS/400 es un ejemplo popular.

La segunda forma es mantener la lista C dentro del sistema operativo. Las capacidades son entonces referidas por su posición en la lista de capacidades. Un proceso podría decir: Leer 1 KB del archivo apuntado por la capacidad 2. Esta forma de direccionamiento es similar al uso de descriptores de archivo en UNIX.

La tercera forma es mantener la lista C en el espacio de usuario, pero gestionar las capacidades criptográficamente para que los usuarios no puedan manipularlas. Este enfoque es especialmente adecuado para los sistemas distribuidos y funciona de la siguiente forma. Cuando un proceso cliente envía un mensaje a un servidor remoto, por ejemplo, un servidor de archivos, para que cree un objeto para él, el servidor crea el objeto y genera un número aleatorio largo, el campo de comprobación, para acompañarlo. Se reserva un espacio en la tabla de archivos del servidor para el objeto y el campo de comprobación se almacena allí junto con las direcciones de los bloques de disco. En términos de UNIX, el campo de comprobación se almacena en el

servidor en el nodo i . No se devuelve al usuario y nunca se pone en la red. El servidor entonces genera y devuelve una capacidad al usuario de la forma mostrada en la siguiente figura.

Servidor	Objeto	Derechos	$F(\text{Objetos, Derechos, Control})$
----------	--------	----------	--

La capacidad devuelta al usuario contiene el identificador del servidor, el número de objeto, el índice en las tablas del servidor, esencialmente, el número de nodo- i , y los derechos, almacenados como un mapa de bits. Para un objeto recién creado, todos los bits de derechos están activados, por supuesto, porque el propietario puede hacer todo. El último campo consiste en la concatenación del objeto, los derechos y el campo de comprobación, que se ejecuta a través de una función unidireccional criptográficamente segura, f . Una función unidireccional criptográficamente segura es una función $y = f(x)$ que tiene la propiedad de que, dado x , es fácil encontrar y , pero dado y , es computacionalmente inviable encontrar x . En otras palabras, basta con saber y que, con una buena función unidireccional, incluso un atacante decidido no podrá adivinar el campo de verificación, aunque conozca todos los demás campos de la capacidad.

Cuando el usuario desea acceder al objeto, envía la capacidad al servidor como parte de la solicitud. El servidor extrae entonces el número de objeto para indexarlo en sus tablas y encontrar el objeto. A continuación, calcula $f(\text{Objeto, Derechos, Control})$, tomando los dos primeros parámetros de la propia capacidad y el tercero de sus propias tablas. Si el resultado coincide con el cuarto campo de la capacidad, se acepta la solicitud; en caso contrario, se rechaza. Si un usuario intenta acceder al objeto de otra persona, no podrá fabricar el cuarto campo correctamente, ya que no conoce el campo de comprobación, y la solicitud será rechazada.

Un usuario puede pedir al servidor que produzca una capacidad más débil, por ejemplo, para un acceso de sólo lectura. En primer lugar, el servidor verifica que la capacidad es válida. Si es así, calcula $f(\text{Objeto, Nuevos derechos, Control})$ y genera una nueva capacidad poniendo este valor en el cuarto campo. Tenga en cuenta que el valor original de Control se utiliza porque otras capacidades pendientes dependen de él.

Esta nueva capacidad se devuelve al proceso solicitante. El usuario puede ahora dársela a un amigo simplemente enviándola en un mensaje. Si el amigo activa bits de derechos que deberían estar desactivados, el servidor lo detectará cuando se utilice la capacidad, ya que el valor f no se corresponderá con el campo de derechos falsos. Como el amigo no conoce el campo de verificación verdadero, no puede fabricar una capacidad que corresponda a los bits de derechos falsos. Este esquema fue desarrollado para el sistema Amoeba.

Además de los derechos específicos dependientes del objeto, como la lectura y la ejecución, las capacidades, tanto las del kernel como las protegidas

criptográficamente, suelen tener derechos genéricos que son aplicables a todos los objetos. Ejemplos de derechos genéricos son:

- Copiar la capacidad: crea una nueva capacidad para el mismo objeto.
- Copiar al objeto: crea un objeto duplicado con una nueva capacidad.
- Eliminar la capacidad: borra una entrada de la lista C; el objeto no se ve afectado.
- Destruir al objeto: elimina permanentemente un objeto y una capacidad.

Una última observación que merece la pena hacerse sobre los sistemas de capacidades es que revocar el acceso a un objeto es bastante difícil en la versión gestionada por el kernel. Es difícil para el sistema encontrar todas las capacidades pendientes de cualquier objeto para recuperarlas, ya que pueden estar almacenadas en listas C por todo el disco. Un enfoque es hacer que cada capacidad apunte a un objeto indirecto, en lugar de al propio objeto. Al hacer que el objeto indirecto apunte al objeto real, el sistema siempre puede romper esa conexión, invalidando así las capacidades. Cuando se presenta una capacidad al objeto indirecto al sistema, el usuario descubrirá que el objeto indirecto está ahora apuntando a un objeto nulo.

En el esquema de Amoeba, la revocación es fácil. Todo lo que hay que hacer es cambiar el campo de verificación almacenado con el objeto. De un golpe, se invalidan todas las capacidades existentes. Sin embargo, ninguno de los dos esquemas permite la revocación selectiva, es decir, retirar, digamos, el permiso de Juan, pero el de nadie más. En general, se reconoce que este defecto es un problema de todos los sistemas de capacidades.

Otro problema general es asegurarse de que el propietario de una capacidad válida no dé una copia a 1000 de sus mejores amigos. Hacer que el kernel gestione las capacidades, como en Hydra, resuelve el problema, pero esta solución no funciona bien en un sistema distribuido como Amoeba.

Resumiendo, muy brevemente, las ACL's y las capacidades tienen propiedades algo complementarias. Las capacidades son muy eficientes porque si un proceso dice: Abrir el archivo apuntado por la capacidad 3, no se necesita ninguna comprobación. Con las ACL's, puede ser necesaria una búsqueda, potencialmente larga, de la ACL. Si no se admiten grupos, conceder a todo el mundo acceso de lectura a un archivo requiere enumerar todos los usuarios en la ACL. Las capacidades también permiten encapsular fácilmente un proceso, mientras que las ACL no. Por otra parte, las ACL permiten la revocación selectiva de derechos, cosa que no hacen las capacidades. Por último, si se elimina un objeto y las capacidades no, o viceversa, surgen problemas. Las ACL no sufren este problema.

La mayoría de los usuarios están familiarizados con las ACL, porque son comunes en sistemas operativos como Windows y UNIX. Sin embargo, las capacidades

tampoco son tan infrecuentes. Por ejemplo, el kernel L4 que se ejecuta en muchos smartphones de muchos fabricantes, normalmente junto a otros sistemas operativos como Android o debajo de ellos, está basado en capacidades. Del mismo modo, el FreeBSD ha adoptado Capsicum, aportando capacidades a un popular miembro de la familia UNIX.

Llave de bloqueo

El esquema de llave de bloqueo es un compromiso entre las listas de control de acceso y las listas de capacidades. Cada objeto tiene una lista de patrones de bits únicos llamados cerraduras. Del mismo modo, cada dominio tiene una lista de patrones de bits únicos llamados claves. Un proceso que se ejecuta en un dominio puede acceder a un objeto sólo si ese dominio tiene una clave que coincide con uno de los bloqueos del objeto. Al igual que con las listas de capacidades, la lista de claves de un dominio debe ser gestionada por el sistema operativo en nombre del dominio. Los usuarios no pueden examinar o modificar la lista de claves, o bloqueos, directamente.

Revocación de los derechos de acceso

En un sistema de protección dinámico, a veces podemos necesitar revocar los derechos de acceso a objetos compartidos por diferentes usuarios. Pueden surgir varias cuestiones sobre la revocación:

- **Inmediata o diferida.** ¿La revocación se produce inmediatamente o se retrasa? Si la revocación se retrasa, ¿podemos saber cuándo tendrá lugar?
- **Selectiva o general.** Cuando se revoca un derecho de acceso a un objeto, ¿afecta a todos los usuarios que tienen un derecho de acceso a ese objeto, o podemos especificar un grupo selecto de usuarios cuyos derechos de acceso deben ser revocados?
- **Parcial frente a total.** ¿Se puede revocar un subconjunto de derechos asociados a un objeto, o hay que revocar todos los derechos de acceso a ese objeto?
- **Temporal o permanente.** ¿Se puede revocar el acceso de forma permanente, es decir, el derecho de acceso revocado no volverá a estar disponible, o se puede revocar el acceso y volver a obtenerlo posteriormente?

Con un esquema de lista de acceso, la revocación es fácil. Se busca en la lista de acceso cualquier derecho de acceso que deba ser revocado, y se elimina de la lista. La revocación es inmediata y puede ser general o selectiva, total o parcial, y permanente o temporal.

Las capacidades, sin embargo, presentan un problema de revocación mucho más difícil, como se ha mencionado anteriormente. Dado que las capacidades

están distribuidas por todo el sistema, debemos encontrarlas antes de poder revocarlas. Los esquemas que implementan la revocación de las capacidades son los siguientes:

- **Readquisición.** Periódicamente, las capacidades se eliminan de cada dominio. Si un proceso quiere usar una capacidad, puede encontrar que esa capacidad ha sido borrada. El proceso puede entonces intentar readquirir la capacidad. Si el acceso ha sido revocado, el proceso no podrá readquirir la capacidad.
- **Apuntadores.** Con cada objeto se mantiene una lista de punteros que apuntan a todas las capacidades asociadas a ese objeto. Cuando se requiere la revocación, podemos seguir estos punteros, cambiando las capacidades según sea necesario. Este esquema fue adoptado en el sistema MULTICS. Es bastante general, pero su implementación es costosa.
- **Indirección.** Las capacidades apuntan indirectamente, no directamente, a los objetos. Cada capacidad apunta a una única entrada en una tabla global, que a su vez apunta al objeto. La revocación se realiza buscando en la tabla global la entrada deseada y borrándola. Entonces, cuando se intenta un acceso, se descubre que la capacidad apunta a una entrada ilegal de la tabla. Las entradas de la tabla pueden reutilizarse para otras capacidades sin dificultad, ya que tanto la capacidad como la entrada de la tabla contienen el nombre único del objeto. El objeto de una capacidad y su entrada en la tabla deben coincidir. Este esquema se adoptó en el sistema CAL. No permite la revocación selectiva.
- **Claves.** Una clave es un patrón de bits único que puede asociarse a una capacidad. Esta clave se define cuando se crea la capacidad, y no puede ser modificada ni inspeccionada por el proceso que posee la capacidad. A cada objeto se le asocia una clave maestra, que puede definirse o sustituirse con la operación set-key. Cuando se crea una capacidad, el valor actual de la clave maestra se asocia a la capacidad. Cuando se ejerce la capacidad, su clave se compara con la clave maestra. Si las claves coinciden, se permite que la operación continúe; en caso contrario, se genera una condición de excepción. La revocación sustituye la clave maestra por un nuevo valor a través de la operación set-key, invalidando todas las capacidades anteriores de este objeto.

Este esquema no permite la revocación selectiva, ya que sólo se asocia una clave maestra a cada objeto. Si asociamos una lista de claves a cada objeto, entonces se puede implementar la revocación selectiva. Por último, podemos agrupar todas las claves en una tabla global de claves. Una capacidad sólo es válida si su clave coincide con alguna clave de la tabla global. La revocación se realiza eliminando de la tabla la clave que coincide. Con este esquema, una clave puede asociarse a varios objetos, y varias

claves pueden asociarse a cada objeto, lo que proporciona la máxima flexibilidad.

En los esquemas basados en claves, las operaciones de definición de claves, de inserción en listas y de eliminación de listas no deberían estar disponibles para todos los usuarios. En particular, sería razonable permitir que sólo el propietario de un objeto establezca las claves de ese objeto. Sin embargo, esta elección es una decisión política que el sistema de protección puede implementar, pero no debe definir.

Control de acceso basado en roles

Ya describimos cómo se pueden utilizar los controles de acceso a los archivos dentro de un sistema de archivos. A cada archivo y directorio se le asigna un propietario, un grupo, o posiblemente una lista de usuarios, y para cada una de esas entidades, se asigna información de control de acceso. Se puede añadir una función similar a otros aspectos de un sistema informático. Un buen ejemplo de esto se encuentra en Solaris 10 y versiones posteriores.

La idea es avanzar en la protección disponible en el sistema operativo añadiendo explícitamente el principio de mínimo privilegio a través del control de acceso basado en roles (RBAC). Esta facilidad gira en torno a los privilegios. Un privilegio es el derecho a ejecutar una llamada al sistema o a utilizar una opción dentro de esa llamada al sistema, como abrir un archivo con acceso de escritura. Los privilegios pueden asignarse a los procesos, limitándolos exactamente al acceso que necesitan para realizar su trabajo. Los privilegios y los programas también se pueden asignar a los roles. A los usuarios se les asignan roles o pueden tomar roles basados en las contraseñas asignadas a los roles. De esta manera, un usuario puede tomar un rol que habilita un privilegio, permitiendo al usuario ejecutar un programa para realizar una tarea específica. Esta implementación de privilegios disminuye el riesgo de seguridad asociado con los superusuarios y los programas setuid. Notemos que esta facilidad es similar a la matriz de acceso ya descrita antes.

Control de acceso obligatorio (MAC)

Los sistemas operativos han utilizado tradicionalmente el control de acceso discrecional (DAC) como medio para restringir el acceso a archivos y otros objetos del sistema. Con DAC, el acceso se controla en base a las identidades de los usuarios individuales o grupos. En los sistemas basados en UNIX, el DAC adopta la forma de permisos de archivo, que se pueden establecer mediante `chmod`, `chown` y `chgrp`, mientras que Windows, y algunas variantes de UNIX, permiten una granularidad más fina mediante listas de control de acceso (ACL).

Sin embargo, las ACL han demostrado ser insuficientes a lo largo de los años. Una debilidad clave reside en su naturaleza discrecional, que permite al

propietario de un recurso establecer o modificar sus permisos. Otro punto débil es el acceso ilimitado que se permite al administrador o usuario root. Como hemos visto, este diseño puede dejar al sistema vulnerable tanto a los ataques accidentales como a los malintencionados y no proporciona ninguna defensa cuando los hackers obtienen privilegios de root.

Por lo tanto, surgió la necesidad de una forma de protección más fuerte, que se introdujo en forma de control de acceso obligatorio (MAC). El MAC se aplica como una política del sistema que ni siquiera el usuario root puede modificar, a menos que la política permita explícitamente las modificaciones o que el sistema se reinicie, normalmente en una configuración alternativa. Las restricciones impuestas por las reglas de la política MAC son más poderosas que las capacidades del usuario root y pueden ser usadas para hacer que los recursos sean inaccesibles para cualquiera que no sea su propietario.

Todos los sistemas operativos modernos proporcionan MAC junto con DAC, aunque las implementaciones difieren. Solaris fue uno de los primeros en introducir MAC, que formaba parte de Trusted Solaris (2.5). Free BSD hizo de DAC parte de su implementación de TrustedBSD (FreeBSD 5.0). La implementación de FreeBSD fue adoptada por Apple en macOS 10.5 y ha servido como sustrato sobre el que se implementan la mayoría de las características de seguridad de MAC e iOS. La implementación de MAC en Linux forma parte del proyecto SELinux, ideado por la NSA, y se ha integrado en la mayoría de las distribuciones. Microsoft Windows se unió a la tendencia con el Control de Integridad Obligatorio de Windows Vista.

En el corazón de MAC está el concepto de etiquetas. Una etiqueta es un identificador, normalmente una cadena, asignado a un objeto, archivos, dispositivos y similares. Las etiquetas también pueden aplicarse a los sujetos (actores, como los procesos). Cuando un sujeto solicita realizar operaciones sobre los objetos. Cuando estas solicitudes deben ser atendidas por el sistema operativo, éste realiza primero las comprobaciones definidas en una política, que dicta si un sujeto con una etiqueta determinada puede o no realizar la operación en el objeto etiquetado.

Como breve ejemplo, consideremos un simple conjunto de etiquetas, ordenadas según el nivel de privilegio: sin clasificar, secreto y alto secreto. Un usuario con autorización secreta podrá crear procesos con etiquetas similares, que tendrán acceso a los archivos no clasificados y secretos, pero no a los archivos de alto secreto. Ni el usuario ni sus procesos serían conscientes de la existencia de archivos de alto secreto, ya que el sistema operativo los filtraría de todas las operaciones de archivo (por ejemplo, no se mostrarían al listar el contenido de los directorios). Los procesos de usuario también estarían protegidos de esta manera, de modo que un proceso no clasificado no podría ver o realizar peticiones IPC a un proceso secreto o de alto secreto. De esta manera, las

etiquetas MAC son una implementación de la matriz de acceso descrita anteriormente.

Seguridad multinivel

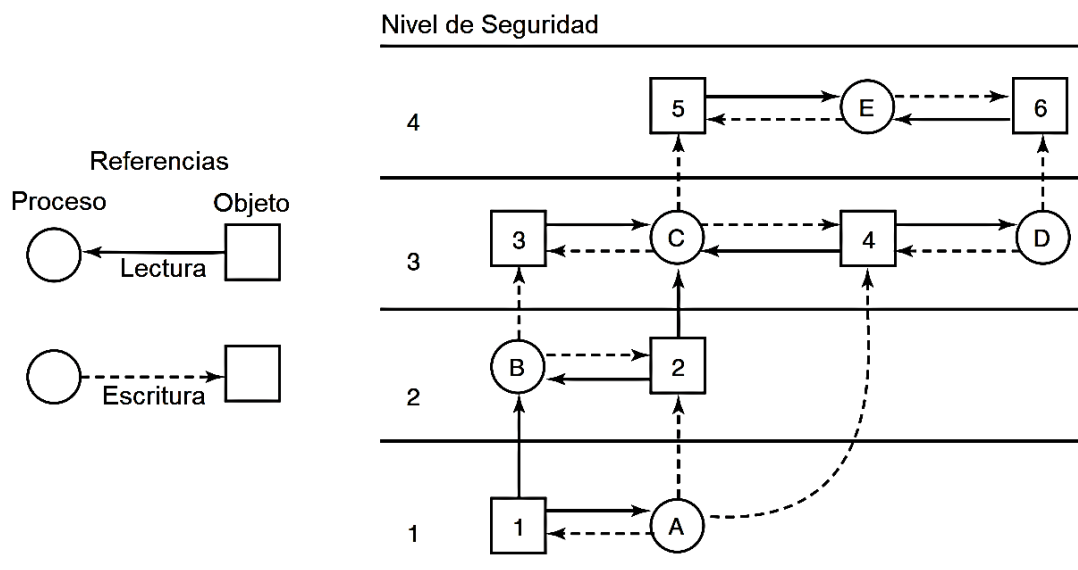
La mayoría de los sistemas operativos permiten a los usuarios individuales determinar quién puede leer y escribir sus archivos y otros objetos. Esta política se llama control de acceso discrecional. En muchos entornos este modelo funciona bien, pero hay otros entornos en los que se requiere una seguridad mucho más estricta, como los militares, los departamentos de patentes de las empresas y los hospitales. En estos últimos entornos, la organización ha establecido reglas sobre quién puede ver qué, y éstas no pueden ser modificadas por soldados, abogados o médicos individuales, al menos no sin obtener un permiso especial del jefe, y probablemente también de los abogados del jefe. Estos entornos necesitan controles de acceso obligatorios para garantizar que el sistema aplique las políticas de seguridad establecidas, además de los controles de acceso discrecionales estándar. Lo que hacen estos controles de acceso obligatorios es regular el flujo de información, para asegurarse de que no se filtre de una manera que no se supone que deba hacerlo.

El modelo de seguridad multinivel más utilizado es el modelo Bell-LaPadula. Este modelo se diseñó para gestionar la seguridad militar, pero también es aplicable a otras organizaciones. En el mundo militar, los documentos, es decir, los objetos, pueden tener un nivel de seguridad, como no clasificado, confidencial, secreto y alto secreto. A las personas también se les asignan estos niveles, en función de los documentos que pueden ver. Un general puede estar autorizado a ver todos los documentos, mientras que un teniente puede estar restringido a los documentos clasificados como confidenciales e inferiores. Un proceso que se ejecuta en nombre de un usuario adquiere el nivel de seguridad de éste. Como hay varios niveles de seguridad, este esquema se denomina sistema de seguridad multinivel.

El modelo Bell-LaPadula tiene reglas sobre cómo puede fluir la información:

- **La propiedad de seguridad simple.** Un proceso que se ejecuta en el nivel de seguridad k sólo puede leer objetos de su nivel o inferior. Por ejemplo, un general puede leer los documentos de un teniente, pero un teniente no puede leer los documentos de un general.
- **La propiedad *.** Un proceso que se ejecuta en el nivel de seguridad k sólo puede escribir objetos de su nivel o superior. Por ejemplo, un teniente puede adjuntar un mensaje al buzón de un general diciendo todo lo que sabe, pero un general no puede adjuntar un mensaje al buzón de un teniente diciendo todo lo que sabe porque el general puede haber visto documentos de alto secreto que no pueden ser revelados a un teniente.

A grandes rasgos, los procesos pueden leer hacia abajo y escribir hacia arriba, pero no a la inversa. Si el sistema hace cumplir rigurosamente estas dos propiedades, se puede demostrar que ninguna información puede filtrarse de un nivel de seguridad superior a otro inferior. La propiedad * se llamó así porque en el informe original, los autores no pudieron pensar en un buen nombre para ella y utilizaron * como marcador temporal hasta que pudieran idear un nombre mejor. Nunca lo hicieron y en el informe se imprimió *. En este modelo, los procesos leen y escriben objetos, pero no se comunican entre sí directamente. El modelo Bell-LaPadula se muestra gráficamente en la figura siguiente.



En esta figura, una flecha sólida que va de un objeto a un proceso indica que el proceso está leyendo el objeto, es decir, la información fluye del objeto al proceso. Del mismo modo, una flecha discontinua de un proceso a un objeto indica que el proceso está escribiendo en el objeto, es decir, la información está fluyendo desde el proceso hacia el objeto. Así, toda la información fluye en la dirección de las flechas. Por ejemplo, el proceso B puede leer del objeto 1 pero no del objeto 3.

La propiedad de seguridad simple dice que todas las flechas sólidas (de lectura) van hacia los lados o hacia arriba. La propiedad * dice que todas las flechas discontinuas, de escritura, también van hacia los lados o hacia arriba. Dado que la información sólo fluye horizontalmente o hacia arriba, cualquier información que comience en el nivel k nunca puede aparecer en un nivel inferior. En otras palabras, nunca hay un camino que mueva la información hacia abajo, lo que garantiza la seguridad del modelo.

El modelo Bell-LaPadula se refiere a la estructura organizativa, pero en última instancia tiene que ser aplicado por el sistema operativo. Una forma de hacerlo es asignar a cada usuario un nivel de seguridad, que se almacenaría junto con otros datos específicos del usuario, como el UID y el GID. Al iniciar la sesión, el

shell del usuario adquiriría el nivel de seguridad del usuario, y éste sería heredado por todos sus hijos. Si un proceso que se ejecuta con un nivel de seguridad k intenta abrir un archivo u otro objeto cuyo nivel de seguridad es superior a k , el sistema operativo debería rechazar el intento de apertura. Del mismo modo, los intentos de abrir cualquier objeto de nivel de seguridad inferior a k para escribir deben fallar.

Para resumir el modelo Bell-LaPadula en términos militares, un teniente puede pedir a un soldado raso que revele todo lo que sabe y luego copiar esta información en el archivo de un general sin violar la seguridad. Ahora pongamos el mismo modelo en términos civiles. Imaginemos una empresa en la que los conserjes tienen un nivel de seguridad 1, los programadores un nivel de seguridad 3 y el presidente de la empresa un nivel de seguridad 5. Utilizando Bell-LaPadula, un programador puede consultar a un conserje sobre los planes futuros de la empresa y luego sobrescribir los archivos del presidente que contienen la estrategia corporativa. Puede que no todas las empresas estén igual de entusiasmadas con este modelo. El problema del modelo Bell-LaPadula es que fue concebido para guardar secretos, no para garantizar la integridad de los datos. Para esto último, necesitamos precisamente las propiedades inversas:

- **La propiedad de integridad simple.** Un proceso que se ejecuta en el nivel de seguridad k sólo puede escribir objetos de su nivel o inferior (no se puede escribir).
- **La propiedad de integridad *.** Un proceso que se ejecuta en el nivel de seguridad k sólo puede leer objetos de su nivel o superior (no se puede leer hacia abajo).

Juntas, estas propiedades garantizan que el programador pueda actualizar los archivos del conserje con la información adquirida del presidente, pero no a la inversa. Por supuesto, algunas organizaciones quieren tanto las propiedades de Bell-LaPadula como las de Biba, pero están en conflicto directo, por lo que son difíciles de conseguir simultáneamente.

Sistemas basados en capacidades

El concepto de protección basada en la capacidad se introdujo a principios de los años 70. Dos de los primeros sistemas de investigación fueron Hydra y CAP. Ninguno de los dos sistemas se utilizó de forma generalizada, pero ambos proporcionaron interesantes campos de prueba para las teorías de protección. Aquí consideramos dos enfoques más contemporáneos de los sistemas basados en capacidades.

Capacidades de Linux

Linux utiliza las capacidades para hacer frente a las limitaciones del modelo UNIX. El grupo de estándares POSIX introdujo las capacidades en POSIX 1003.1e.

Aunque POSIX.1e fue finalmente retirado, Linux se apresuró a adoptar las capacidades en la versión 2.2 y ha continuado añadiendo nuevos desarrollos.

En esencia, las capacidades de Linux dividen los poderes de root en áreas distintas, cada una representada por un bit en una máscara de bits. Se puede lograr un control detallado sobre las operaciones privilegiadas alternando los bits de la máscara de bits. En la práctica, se utilizan tres máscaras de bits, que indican las capacidades permitidas, efectivas y heredables. Las máscaras de bits pueden aplicarse por proceso o por hilo. Además, una vez revocadas, las capacidades no pueden volver a adquirirse. La secuencia habitual de eventos es que un proceso o hilo comienza con el conjunto completo de capacidades permitidas y disminuye voluntariamente ese conjunto durante la ejecución. Por ejemplo, después de abrir un puerto de red, un hilo puede eliminar esa capacidad para que no se puedan abrir más puertos.

Probablemente pueda ver que las capacidades son una implementación directa del principio de privilegio mínimo. Como ya explicamos, este principio de seguridad dicta que a una aplicación o usuario se le deben dar sólo aquellos derechos que se requieren para su funcionamiento normal.

Android, que se basa en Linux, también utiliza capacidades, que permiten a los procesos del sistema, en particular el servidor del sistema, evitar la propiedad de root, en lugar de permitir selectivamente sólo las operaciones necesarias.

El modelo de capacidades de Linux es una gran mejora respecto al modelo tradicional de UNIX, pero sigue siendo inflexible. Por un lado, el uso de un mapa de bits con un bit que representa cada capacidad hace imposible añadir capacidades dinámicamente y requiere recompilar el kernel para añadir más. Además, la función sólo se aplica a las capacidades forzadas por el kernel.

Derechos de Darwin

La protección del sistema de Apple adopta la forma de derechos. Los derechos son una lista de propiedades XML de permisos declarados que indican qué permisos son reclamados como necesarios por el programa. Cuando el proceso intenta realizar una operación privilegiada, se comprueban sus derechos, y sólo si los derechos necesarios están presentes se permite la operación.

Para evitar que los programas reclamen arbitrariamente un derecho, Apple incorpora los derechos en la firma del código. Una vez cargado, un proceso no tiene forma de acceder a su firma de código. Otros procesos, y el kernel, pueden consultar fácilmente la firma, y en particular los derechos. Por lo tanto, la verificación de un derecho es una simple operación de coincidencia de cadenas. De este modo, sólo las aplicaciones verificables y autenticadas pueden reclamar derechos. Todos los derechos del sistema (com.apple.*) están además restringidos a los propios binarios de Apple.

Otros métodos de mejora de la protección

A medida que aumenta la batalla para proteger los sistemas de daños accidentales y malintencionados, los diseñadores de sistemas operativos están implementando más tipos de mecanismos de protección en más niveles. En esta sección se examinan algunas mejoras importantes de la protección en el mundo real.

Protección de la integridad del sistema

Apple introdujo en macOS 10.11 un nuevo mecanismo de protección llamado System Integrity Protection (SIP). Los sistemas operativos basados en Darwin utilizan SIP para restringir el acceso a los archivos y recursos del sistema de forma que ni siquiera el usuario root pueda manipularlos. SIP utiliza atributos extendidos en los archivos para marcarlos como restringidos y protege aún más los binarios del sistema para que no puedan ser depurados o escrutados, y mucho menos manipulados. Lo más importante es que sólo se permiten las extensiones del kernel firmadas por el código, y SIP puede configurarse además para permitir también sólo los binarios firmados por el código.

Bajo SIP, aunque el usuario root sigue siendo el más poderoso del sistema, puede hacer mucho menos que antes. El usuario root todavía puede gestionar los archivos de otros usuarios, así como instalar y eliminar programas, pero no podrá reemplazar o modificar los componentes del sistema operativo. El SIP se implementa como una pantalla global e ineludible en todos los procesos, con las únicas excepciones permitidas para los binarios del sistema, por ejemplo, fsck o kextload, que están específicamente habilitados para operaciones para su propósito designado.

Filtrado de llamadas al sistema

Recordemos que los sistemas monolíticos colocan toda la funcionalidad del kernel en un único archivo que se ejecuta en un único espacio de direcciones. Comúnmente, los kernels de sistemas operativos de propósito general son monolíticos, y por lo tanto se confía implícitamente en ellos como seguros. El límite de confianza, por tanto, se encuentra entre el modo kernel y el modo usuario, en la capa del sistema. Podemos asumir razonablemente que cualquier intento de comprometer la integridad del sistema se hará desde el modo de usuario por medio de una llamada al sistema. Por ejemplo, un atacante puede intentar obtener acceso explotando una llamada al sistema sin protección.

Por lo tanto, es imperativo implementar alguna forma de filtrado de llamadas al sistema. Para lograrlo, podemos añadir código al kernel para realizar una inspección en la puerta de llamadas al sistema, restringiendo una llamada a un subconjunto de llamadas al sistema consideradas seguras o necesarias para la función de esa persona. Se pueden crear perfiles específicos de llamadas al

sistema para procesos individuales. El mecanismo de Linux SECCOMP-BPF hace precisamente eso, aprovechando el lenguaje Berkeley Packet Filter para cargar un perfil personalizado a través de la llamada al sistema `prctl`, propiedad de Linux. Este filtrado es voluntario, pero puede aplicarse eficazmente si se llama desde una biblioteca en tiempo de ejecución cuando se inicializa o desde el propio cargador antes de transferir el control al punto de entrada del programa.

Una segunda forma de filtrado de llamadas al sistema va más allá e inspecciona los argumentos de cada llamada al sistema. Esta forma de protección se considera mucho más fuerte, ya que incluso las llamadas al sistema aparentemente benignas pueden albergar graves vulnerabilidades. Este fue el caso de la llamada al sistema `fast mutex` (`futex`) de Linux. Una condición de carrera en su implementación condujo a una sobreescritura de la memoria del kernel controlada por un atacante y al compromiso total del sistema. Los mutex son un componente fundamental de la multitarea, por lo que la propia llamada al sistema no podía filtrarse por completo.

Uno de los retos encontrados con ambos enfoques es mantenerlos tan flexibles como sea posible y, al mismo tiempo, evitar la necesidad de reconstruir el kernel cuando se requieren cambios o nuevos filtros, algo común debido a las diferentes necesidades de los distintos procesos. La flexibilidad es especialmente importante dada la naturaleza impredecible de las vulnerabilidades. Cada día se descubren nuevas vulnerabilidades que pueden ser explotadas inmediatamente por los atacantes.

Una forma de afrontar este reto es desvincular la implementación del filtro del propio kernel. El kernel sólo tiene que contener un conjunto de llamadas, que pueden implementarse en un controlador especializado (Windows), un módulo del kernel (Linux) o una extensión (Darwin). Como un componente externo y modular proporciona la lógica de filtrado, puede ser actualizado de forma independiente respecto del kernel. Este componente suele hacer uso de un lenguaje especializado en la elaboración de perfiles, incluyendo un intérprete o analizador incorporado. De este modo, el propio perfil puede desvincularse del código, proporcionando un perfil editable y legible por el ser humano y simplificando aún más las actualizaciones. También es posible que el componente de filtrado llame en modo usuario a un proceso demonio de confianza para ayudar con la lógica de validación.

Sandboxing

El sandboxing consiste en ejecutar procesos en entornos que limitan lo que pueden hacer. En un sistema básico, un proceso se ejecuta con las credenciales del usuario que lo inició y tiene acceso a todas las cosas a las que el usuario puede acceder. Si se ejecuta con privilegios del sistema, como el de `root`, el proceso puede hacer literalmente cualquier cosa en el sistema. Casi siempre se da el caso de que un proceso no necesita privilegios completos de usuario o de

sistema. Por ejemplo, ¿necesita un procesador de textos aceptar conexiones de red? ¿Necesita un servicio de red que proporciona la hora del día acceder a archivos más allá de un conjunto específico?

El término sandboxing se refiere a la práctica de imponer limitaciones estrictas a un proceso. En lugar de dar a ese proceso el conjunto completo de llamadas al sistema que le permitirían sus privilegios, imponemos un conjunto inamovible de restricciones al proceso en las primeras etapas de su inicio -bastante antes de la ejecución de su función `main()` y a menudo tan pronto como su creación con la llamada al sistema `fork`. El proceso es entonces incapaz de realizar cualquier operación fuera de su conjunto permitido. De esta manera, es posible evitar que el proceso se comuniquen con cualquier otro componente del sistema, lo que resulta en una compartimentación estricta que mitiga cualquier daño al sistema incluso si el proceso se ve comprometido.

Existen numerosos enfoques para el sandboxing. Java y .net, por ejemplo, imponen restricciones de sandbox a nivel de la máquina virtual. Otros sistemas imponen el sandboxing como parte de su política de control de acceso obligatorio (MAC). Un ejemplo es Android, que se basa en una política SELinux mejorada con etiquetas específicas para las propiedades del sistema y los puntos finales de servicio.

El sandboxing también puede implementarse como una combinación de múltiples mecanismos. Para Android, SELinux ha sido útil, pero con carencias, ya que no puede restringir eficazmente las llamadas individuales del sistema. Las últimas versiones de Android (Nougat y O) utilizan un mecanismo subyacente de Linux llamado SECCOMP-BPF, para aplicar restricciones a las llamadas del sistema mediante el uso de una llamada al sistema especializada. La biblioteca de tiempo de ejecución C en Android (Bionic) llama a esta llamada del sistema para imponer restricciones a todos los procesos de Android y a las aplicaciones de terceros.

Entre los principales proveedores, Apple fue el primero en implementar el sandboxing, que apareció en macOS 10.5 (Tiger) como Seatbelt. El cinturón de seguridad era optativo en lugar de obligatorio, permitiendo, pero no exigiendo, a las aplicaciones que lo utilizaran. El sandbox de Apple se basaba en perfiles dinámicos escritos en el lenguaje Scheme, que permitían controlar no sólo las operaciones que se permitían o bloqueaban, sino también sus argumentos. Esta capacidad permitió a Apple crear diferentes perfiles personalizados para cada binario del sistema, una práctica que continúa hasta hoy.

El sandboxing de Apple ha evolucionado considerablemente desde su creación. Ahora se utiliza en las variantes de iOS, donde sirve, junto con la firma de código, como la principal protección contra el código de terceros no fiable. En iOS, y a partir de macOS 10.8, el sandbox de macOS es obligatorio y se aplica automáticamente a todas las aplicaciones descargadas de la tienda de Mac. Más

recientemente, como ya se ha mencionado, Apple adoptó la Protección de la Integridad del Sistema (SIP), utilizada en macOS 10.11 y posteriores. SIP es, en efecto, un perfil de plataforma para todo el sistema. Apple lo aplica a partir del arranque del sistema en todos los procesos de este. Sólo los procesos que tienen derecho a ello pueden realizar operaciones privilegiadas, y éstas están firmadas por Apple y, por tanto, son de confianza.

Firma de código

En un nivel fundamental, ¿cómo puede un sistema confiar en un programa o script? Por lo general, si el elemento vino como parte del sistema operativo, debería ser de confianza. ¿Pero qué pasa si el elemento se modifica? Si ha sido modificado por una actualización del sistema, entonces también es de confianza, pero en caso contrario no debería ser ejecutable o debería requerir un permiso especial, sea del usuario o del administrador, antes de ser ejecutado. Las herramientas de terceros, comerciales o no, son más difíciles de juzgar. ¿Cómo podemos estar seguros de que la herramienta no ha sido modificada en su camino desde el lugar donde fue creada hasta nuestros sistemas?

Actualmente, la firma de código es la mejor herramienta del arsenal de protección para resolver estos problemas. La firma de código es la firma digital de programas y ejecutables para confirmar que no han sido modificados desde que el autor los creó. Utiliza un hash criptográfico para comprobar la integridad y la autenticidad. La firma de código se utiliza tanto para distribuciones de sistemas operativos como para parches y herramientas de terceros. Algunos sistemas operativos, como iOS, Windows y macOS, se niegan a ejecutar programas que no superan la comprobación de firma de código. También puede mejorar la funcionalidad del sistema de otras maneras. Por ejemplo, Apple puede inhabilitar todos los programas escritos para una versión ya obsoleta de iOS deteniendo su firma de esos programas cuando se descargan del App Store.

Protección basada en el lenguaje

En la medida en que la protección se proporciona en los sistemas informáticos, generalmente se logra a través del kernel del sistema operativo, que actúa como un agente de seguridad para inspeccionar y validar cada intento de acceder a un recurso protegido. Dado que la validación exhaustiva de los accesos puede ser una fuente de sobrecarga considerable, o bien hay que darle soporte de hardware para reducir el costo de cada validación, o bien hay que permitir que el diseñador del sistema comprometa los objetivos de la protección. Satisfacer todos estos objetivos es difícil si la flexibilidad para aplicar las políticas de protección está restringida por los mecanismos de soporte proporcionados o si los entornos de protección se hacen más grandes de lo necesario para garantizar una mayor eficiencia operativa.

A medida que los sistemas operativos se han ido haciendo más complejos, y sobre todo cuando han intentado ofrecer interfaces de usuario de mayor nivel, los objetivos de la protección se han vuelto mucho más refinados. Los diseñadores de sistemas de protección se han basado en gran medida en ideas originadas en los lenguajes de programación y, especialmente, en los conceptos de tipos de datos abstractos y objetos. Los sistemas de protección se preocupan ahora no sólo por la identidad de un recurso al que se intenta acceder, sino también por la naturaleza funcional de ese acceso. En los sistemas de protección más recientes, la preocupación por la función que se invoca va más allá de un conjunto de funciones definidas por el sistema, como los métodos estándar de acceso a archivos, para incluir también funciones que pueden ser definidas por el usuario.

Las políticas de uso de los recursos también pueden variar, dependiendo de la aplicación, y pueden estar sujetas a cambios con el tiempo. Por estas razones, la protección no puede seguir considerándose un asunto que concierne únicamente al diseñador de un sistema operativo. También debe estar disponible como una herramienta para ser utilizada por el diseñador de la aplicación, de modo que los recursos de un subsistema de aplicación puedan ser protegidos contra la manipulación o la influencia de un error.

En este punto, los lenguajes de programación entran en escena. Especificar el control deseado de acceso a un recurso compartido en un sistema es hacer una declaración sobre el recurso. Este tipo de declaración puede integrarse en un lenguaje mediante una extensión de su facilidad de tipificación. Cuando la protección se declara junto con la tipificación de datos, el diseñador de cada subsistema puede especificar sus requisitos de protección, así como su necesidad de uso de otros recursos en un sistema.

Dicha especificación debe darse directamente cuando se compone un programa, y en el lenguaje en el que se declara el propio programa. Este enfoque tiene varias ventajas significativas:

- Las necesidades de protección se declaran simplemente, en lugar de programarse como una secuencia de llamadas a procedimientos de un sistema operativo.
- Los requisitos de protección pueden declararse independientemente de las facilidades que ofrezca un sistema operativo concreto.
- No es necesario que el diseñador de un subsistema proporcione los medios para su cumplimiento.
- Una notación declarativa es natural porque los privilegios de acceso están estrechamente relacionados con el concepto lingüístico de tipo de datos.

La implementación de un lenguaje de programación puede proporcionar una variedad de técnicas para reforzar la protección, pero cualquiera de ellas debe

depender de cierto grado de apoyo de una máquina subyacente y su sistema operativo. Por ejemplo, supongamos que se utiliza un lenguaje para generar código que se ejecuta en el sistema Cambridge CAP (Sección A.14.2). En este sistema, cada referencia de almacenamiento hecha en el hardware subyacente ocurre indirectamente a través de una capacidad. Esta restricción impide que cualquier proceso acceda a un recurso fuera de su entorno de protección en cualquier momento. Sin embargo, un programa puede imponer restricciones arbitrarias sobre cómo se puede utilizar un recurso durante la ejecución de un segmento de código concreto.

La forma más fácil de implementar estas restricciones es utilizando las capacidades de software que proporciona CAP. Una implementación del lenguaje podría proporcionar procedimientos protegidos estándar para interpretar las capacidades del software que realizarían las políticas de protección que podrían especificarse en el lenguaje. Este esquema pone la especificación de la política a disposición de los programadores, al tiempo que los libera de implementar su cumplimiento.

Aunque un sistema no proporcione un kernel de protección tan potente como los de Hydra o CAP, siguen existiendo mecanismos para implementar las especificaciones de protección dadas en un lenguaje de programación. La principal diferencia es que la seguridad de esta protección no será tan grande como la soportada por un kernel de protección, porque el mecanismo debe basarse en más suposiciones sobre el estado operativo del sistema. Un compilador puede separar las referencias para las que puede certificar que no podría producirse una violación de la protección de aquellas para las que podría producirse una violación, y puede tratarlas de forma diferente. La seguridad que proporciona esta forma de protección se basa en la suposición de que el código generado por el compilador no será modificado antes o durante su ejecución. Entonces, ¿cuáles son las ventajas relativas de la aplicación basada únicamente en un kernel, frente a la aplicación proporcionada en gran medida por un compilador? Veámoslas:

- **Seguridad.** La aplicación por parte de un kernel proporciona un mayor grado de seguridad del propio sistema de protección que la generación de código de comprobación de la protección por parte de un compilador. En un esquema soportado por el compilador, la seguridad se basa en la corrección del traductor, en algún mecanismo subyacente de gestión del almacenamiento que protege los segmentos desde los que se ejecuta el código compilado y, en última instancia, en la seguridad de los archivos desde los que se carga un programa. Algunas de estas consideraciones también se aplican a un kernel de protección soportado por software, pero en menor grado, ya que el kernel puede residir en segmentos de almacenamiento físico fijos y puede cargarse sólo desde un archivo designado. Con un sistema de capacidad etiquetada, en el que todo el cálculo

de direcciones se realiza por hardware o por un microprograma fijo, es posible una seguridad aún mayor. La protección soportada por hardware también es relativamente inmune a las violaciones de la protección que podrían ocurrir como resultado de un mal funcionamiento del hardware o del software del sistema.

- **Flexibilidad.** La flexibilidad de un kernel de protección a la hora de implementar una política definida por el usuario es limitada, aunque puede ofrecer facilidades adecuadas para que el sistema pueda aplicar sus propias políticas. Con un lenguaje de programación, la política de protección puede ser declarada, y la aplicación proporcionada según sea necesario por una implementación. Si un lenguaje no proporciona suficiente flexibilidad, puede ser ampliado o sustituido con menos problemas que los que causaría la modificación de un kernel del sistema operativo.
- **Eficiencia.** La mayor eficiencia se obtiene cuando la aplicación de la protección es soportada directamente por el hardware (o microcódigo). En la medida en que se requiera el apoyo del software, la aplicación basada en el lenguaje tiene la ventaja de que la aplicación del acceso estático puede verificarse fuera de línea en tiempo de compilación. Además, dado que un compilador inteligente puede adaptar el mecanismo de aplicación para satisfacer la necesidad especificada, a menudo se puede evitar la sobrecarga fija de las llamadas al kernel.

En resumen, la especificación de la protección en un lenguaje de programación permite la descripción de alto nivel de las políticas de asignación y uso de los recursos. Una implementación del lenguaje puede proporcionar un software para el cumplimiento de la protección cuando no está disponible la comprobación automática soportada por el hardware. Además, puede interpretar las especificaciones de protección para generar llamadas a cualquier sistema de protección proporcionado por el hardware y el sistema operativo.

Una forma de hacer que la protección esté disponible para el programa de aplicación es mediante el uso de una capacidad de software que podría utilizarse como objeto de cálculo. Inherente a este concepto está la idea de que ciertos componentes del programa podrían tener el privilegio de crear o examinar estas capacidades de software. Un programa creador de capacidades podría ejecutar una operación primitiva que sellara una estructura de datos, haciendo que su contenido fuera inaccesible para cualquier componente del programa que no tuviera el privilegio de sellar o desellar. Estos componentes podrían copiar la estructura de datos o pasar su dirección a otros componentes del programa, pero no podrían acceder a su contenido. La razón para introducir estas capacidades de software es introducir un mecanismo de protección en el lenguaje de programación. El único problema con el concepto propuesto es que el uso de las operaciones de sellado y desellado adopta un enfoque

procedimental para especificar la protección. Una notación no procedimental o declarativa parece una forma preferible de poner la protección a disposición del programador de aplicaciones.

Lo que se necesita es un mecanismo seguro y dinámico de control de acceso para distribuir las capacidades de los recursos del sistema entre los procesos de los usuarios. Para contribuir a la fiabilidad general de un sistema, el mecanismo de control de acceso debe ser seguro de usar. Para ser útil en la práctica, también debe ser razonablemente eficiente. Este requisito ha llevado al desarrollo de una serie de construcciones de lenguaje que permiten al programador declarar varias restricciones en el uso de un recurso gestionado específico. Estas construcciones proporcionan mecanismos para tres funciones:

- Distribuir las capacidades de forma segura y eficiente entre los procesos del cliente. En particular, los mecanismos garantizan que un proceso de usuario utilizará el recurso gestionado sólo si se le concedió una capacidad para ese recurso.
- Especificar el tipo de operaciones que un proceso concreto puede invocar sobre un recurso asignado. Por ejemplo, a un lector de un archivo sólo se le debe permitir leer el archivo, mientras que un escritor debe poder tanto leer como escribir. No debería ser necesario conceder el mismo conjunto de derechos a cada proceso de usuario, y debería ser imposible para un proceso ampliar su conjunto de derechos de acceso, excepto con la autorización del mecanismo de control de acceso.
- Especificar el orden en que un proceso concreto puede invocar las distintas operaciones de un recurso. Por ejemplo, un archivo debe abrirse antes de poder leerse. Debería ser posible dar a dos procesos diferentes restricciones sobre el orden en que pueden invocar las operaciones del recurso asignado.

La incorporación de los conceptos de protección en los lenguajes de programación, como herramienta práctica para el diseño de sistemas, está en sus inicios. Es probable que la protección se convierta en una cuestión que preocupe más a los diseñadores de nuevos sistemas con arquitecturas distribuidas y requisitos cada vez más estrictos sobre la seguridad de los datos. Entonces se reconocerá más ampliamente la importancia de las notaciones de lenguaje adecuadas para expresar los requisitos de protección.

ITIL

Como corolario de todo lo visto sobre seguridad y protección resulta apropiado hacer un breve comentario sobre las herramientas mencionadas en ITIL. Sin embargo, a esta altura del apunte, cabe preguntarse ¿qué es ITIL, para qué sirve, qué beneficios tiene, etc.? ITIL es un acrónimo de Biblioteca de Infraestructura de Tecnologías de la Información (ITIL: Information Technology

Infrastructure Library). A pesar de que este largo nombre aún existe oficialmente, ahora, en general, se la denomina simplemente ITIL.

El gobierno británico creó ITIL cuando observó que la creciente dependencia en TI requería un conjunto de prácticas estándar. La norma ahora está publicada y pertenece a una organización conjunta entre una empresa privada, Capita, y la Oficina del Gabinete del Reino Unido, denominada Axelos. La definición oficial de ITIL es:

Los estándares que se presentan en este programa permiten que la empresa pueda tomar el control absoluto de la gestión de sus recursos a través de la reestructuración de sus procesos. Todo ello facilitará la identificación de fallas y carencias en los procesos, de manera que se puedan aplicar acciones para contrarrestar estas deficiencias.

La Biblioteca de Infraestructura de Tecnologías de Información es un conjunto de conceptos y guía de buenas prácticas usadas para la gestión de servicios de tecnologías de la información, el desarrollo de tecnologías de la información y las operaciones relacionadas con la misma en general. Esta guía ha sido elaborada para abarcar toda la infraestructura, desarrollo y operaciones de TI y gestionarla hacia la mejora de la calidad del servicio y es utilizada para definir una metodología de gestión a través de la cual se plantean ciertas prácticas estandarizadas cuyo objetivo es optimizar la prestación de un servicio. Es decir, la idea de este programa es reorganizar los procesos de la empresa para mejorar los resultados.

Además, esto permite a las organizaciones y a los individuos prestar una gestión de servicios de IT rentable, alinear su gestión con la visión, estrategia y crecimiento empresariales, y actuar como un punto único de contacto entre el proveedor de servicios y los usuarios finales. Si bien la gestión de servicios de IT (ITSM) ayuda a las empresas a lograr su misión y visión gracias a la adecuada combinación de procesos, tecnología y personas, el marco de trabajo ITIL guía la forma de gestionar y prestar los servicios. Las iniciativas de tecnología y transformación digital se guían a través de las mejores prácticas del marco de trabajo ITIL y las empresas lo adoptan para conseguir más rápidamente sus beneficios comerciales gracias a procesos definidos y posibles mediante la tecnología adecuada.

Resulta importante tener en cuenta que este programa abarca una serie de conceptos y herramientas que facilitan la gestión de la prestación de servicios, especialmente aquellos relacionados con las tecnologías de la información y la comunicación. ITIL no es un plan totalmente estricto; es un programa que se adapta a las necesidades de la compañía, por lo tanto, puede implementarse de acuerdo con los lineamientos y objetivos de ella.

El conjunto de procesos ITIL, desde su creación en la década de 1980 por la Agencia Central de Ordenadores y Telecomunicaciones (CCTA) del gobierno

británico, ha evolucionado continuamente en muchas versiones y, en la actualidad, cuenta con 5 libros. A lo largo de los años, ha sido objeto de varias versiones orientadas a la alineación con las empresas y a una mejora de la eficiencia. El primer libro del marco de trabajo ITIL se publicó en el año 1989, seguido de la versión 2 en el 2000. ITIL v3 se publicó en el año 2007 y una variante más actualizada de esta versión, ITIL v4, se lanzó en el año 2011.

Esta cuarta versión fue nuevamente actualizada a comienzos del 2019 con el fin de mostrar los cambios de la Gestión de Servicios de TI, dado que la era tecnológica y de transformación digital en la que vivimos convierte la gestión de TI en un desafío cada vez más grande para adaptarse a las nuevas necesidades de las organizaciones. Esta cuarta versión responde a los nuevos enfoques de gestión de servicios e identifica Agile, DevOps y Lean como nuevas prácticas emergentes compenetradas con ITIL. En este desarrollo se pueden apreciar los siguientes principios son muy significativos dentro del marco establecido por ITIL:

- Procesos, necesarios para la gestión de TI de acuerdo con la alineación de estos dentro de la organización.
- Calidad, entendida como la entrega a cliente del producto o servicio óptimos, es decir, incluyendo las características acordadas.
- Cliente, su satisfacción es el objetivo de la mejora de los servicios, siendo, por lo tanto, el beneficiario directo de la implantación de las buenas prácticas de ITIL.
- Independencia, siempre deben mantenerse buenas prácticas a pesar de los métodos establecidos para cada proceso y de los proveedores existentes.

ITIL es uno de los marcos de referencia más utilizados del mundo y debido a su incisión en la entrega de servicios eficiente, ITIL se define como Sistema de Valores de Servicio (SVS). Estos valores son las actividades y componentes de las organizaciones que a través del trabajo conjunto crean valor. Estos valores pueden ser tanto para las organizaciones como para sus clientes y el resto de las partes interesadas. El marco de trabajo ITIL consta de 5 etapas que reflejan el ciclo de vida del servicio. Cada etapa abarca un conjunto de procesos o funciones que se alinean con la estructura organizacional informática. Las empresas adoptan algunos de estos procesos para que sean adecuados para sus equipos. Por lo tanto, el marco de trabajo ITIL es flexible en términos de su adopción. A continuación, tenemos las etapas del ciclo de vida del servicio del marco de trabajo ITIL:

- **Estrategia.** Se enfoca en el mejoramiento de la gestión a través de la implementación de estrategias empresariales alineadas con el propósito de la organización.

- **Diseño.** Se trata de la creación de métodos para transformar los procesos referentes a la prestación del servicio.
- **Transición.** Es el tiempo en que se aplican los cambios para transformar los procesos.
- **Operación.** Se refiere a la aplicación de ciertas prácticas que permiten el mejoramiento de los procedimientos rutinarios.
- **Mejora continua.** Aquí, la idea es aplicar estrategias para ofrecer valor agregado a los servicios e innovar en cada proceso.

Los fundamentos de ITIL presentados anteriormente se deben complementar con unos conceptos clave que ilustran su importancia y que presentamos a continuación:

- **Gestión del cambio.** A diario, las empresas instrumentan nuevos proyectos o aplicaciones. Sin embargo, es importante analizar los riesgos y los efectos antes de llevar a cabo las actividades planificadas. En función de los riesgos y los efectos, las aprobaciones de los cambios se planifican, incluyendo a todos los interesados pertinentes. La gestión del cambio impulsada por ITIL registra todos los detalles de cambio que se soliciten para controlar y auditar de la mejor manera. Por tanto, la gestión del cambio es crucial para una organización a fin de implementar eficazmente las nuevas instrumentaciones sin ninguna interrupción. Una revisión posterior a la instrumentación se realiza para mantener todo bajo control.
- **Gestión de incidencias.** Los usuarios de las empresas esperan unas operaciones comerciales ininterrumpidas y sin ninguna perturbación. Sin embargo, esto resulta todo un desafío considerando la complejidad de las aplicaciones y los componentes de infraestructura disponibles. Un enfoque proactivo de la gestión de incidencias facilitado por ITIL desvía los tickets de asistencia redundantes y también elimina la ocurrencia de incidencias graves. La automatización se ocupa de la clasificación y asignación de los tickets de asistencia, de manera que los agentes del Service Desk puedan centrarse en las actividades más prioritarias. La gestión de incidencias del marco de trabajo ITIL trabaja en estrecha colaboración con la base de datos de gestión de la configuración (CMDB) para vincular el activo relacionado con el incidente.
- **Gestión de problemas.** El equipo de gestión de problemas se encarga de las incidencias repetitivas para analizar la causa raíz y encontrar una solución permanente. El responsable de la gestión de los problemas realiza el análisis de causa raíz y sugiere una solución temporal hasta que se implemente una solución permanente. Gracias a ITIL, muchas organizaciones adoptan la gestión proactiva de los problemas para evitar cualquier daño potencial ocasionado por interrupciones importantes y garantizar la disponibilidad de los servicios. Se recomienda comprender la diferencia entre la gestión de

problemas y la gestión de incidencias a fin de asignar roles y responsabilidades claros.

Objetivos de ITIL

El primer paso consiste en comprender los problemas que intentamos resolver de modo que se puedan alinear los objetivos empresariales con los objetivos de ITIL. Centrémonos en las prioridades de los clientes y comuniquemos el valor de ITIL a todos los jefes de departamentos, de manera que toda la organización esté concientizada. De esta manera entenderán que su objetivo consiste en añadir valor real a su inversión en las tecnologías de la información y monitorizar la infraestructura y las operaciones. Estos conceptos los podemos representar con lo siguiente:

- **Analizar el proceso actual.** Antes de implementar el marco de trabajo ITIL, analicemos la madurez del proceso actual para identificar las áreas de mejora. De alguna forma, ya se podrían estar utilizando ciertos aspectos de ITIL. En función de las prioridades de la empresa, cerremos la brecha adoptando los procesos del marco de trabajo ITIL pertinentes. La evaluación de la madurez de los procesos debe ser realista para conseguir los resultados deseados.
- **Definir los procesos ITIL y su propietario.** Las personas son más importantes que los procesos y las capacidades tecnológicas. Por ello, la identificación de las funciones y responsabilidades del propietario del proceso es crucial. El éxito del marco de trabajo ITIL depende en gran medida de la actitud y las habilidades sociales, además de las técnicas, de los agentes del Service Desk. Asimismo, ITIL exige una gran colaboración entre las diversas funciones y, por lo tanto, las capacidades de las personas son importantes a la hora de contratar agentes para el Service Desk.
- **Comenzar a implementar los procesos.** El marco de trabajo ITIL no obliga a las empresas a implementar la totalidad de los 26 procesos; más bien se recomienda comenzar con los procesos necesarios, como la gestión de incidencias y la gestión del conocimiento, e ir avanzando con otros procesos tras haber tenido un éxito inicial. Comuniquemos el marco de trabajo ITIL y las capacidades de proceso a todos los interesados pertinentes.
- **Revisar y mejorar.** La mejora continua del servicio (CSI, por sus siglas en inglés) es necesaria desde un punto de vista operativo. La revisión de la madurez de los procesos en curso ayuda a identificar las brechas de los procesos. La mejora continua del servicio ocurre a lo largo del ciclo de vida de ITIL, ya que este es un proyecto en curso que se revisa y mejora de principio a fin. Esto también ayuda a alinear los objetivos empresariales con los objetivos del marco de trabajo ITIL.

Cabe destacar que, para enfocar este camino hacia la calidad del servicio, es necesario contar con información fiable y segura, ya que es imprescindible que ésta sea completa y precisa para la toma de decisiones.

Beneficios de ITIL

El marco de trabajo y la metodología ITIL supone variados beneficios para las tecnologías de la información y las empresas. Por un lado, posibilita que las empresas sean más ágiles al complementar otras mejores prácticas y marcos de trabajo. Además, actúa como un controlador para una gestión eficiente de los servicios de TI. A continuación, se indican algunos de los beneficios más en detalle:

- **Mejora en la calidad de servicio.** Garantiza la consistencia en los niveles del servicio y mejora la eficiencia gracias a una mayor calidad del servicio.
- **Optimización de costos.** La optimización de los costes se puede lograr mediante la priorización de los recursos y su uso eficiente en función de los requisitos de las empresas y de los clientes.
- **Mayor nivel de satisfacción de los clientes.** La mejora de la calidad del servicio aumenta la satisfacción de los clientes en el futuro, ya que ITIL entiende los requisitos de los clientes.
- **Mejor gestión del riesgo.** El marco de trabajo ITIL contribuye a la gestión proactiva de los servicios de TI, evitando incidencias graves y manteniendo el riesgo bajo control. Gracias a la más reciente gestión de servicios de TI que emplea inteligencia artificial (IA), los eventos futuros se pueden predecir en función de las tendencias pasadas.
- **Alineación de los objetivos empresariales.** Optimice los artículos de la base de conocimientos para los motores de búsqueda de manera que los usuarios puedan encontrar fácilmente las soluciones.

Contar con una certificación en ITIL trae muchos beneficios para el desarrollo de la empresa, como ser: aumento de la productividad y mejora de la gestión de recursos, mejor comprensión de los clientes para poder ofrecerles servicios que se adapten a sus necesidades, asegurar una mayor alineación entre TI y otras áreas de la empresa, disminución de riesgos, entre otros.

Estructura de las prácticas incluidas

Además de los procesos, ITIL incluye diversas prácticas:

- Prácticas generales de gestión: estratégica, gestión de riesgos y de la mejora continua.
- Prácticas de gestión de servicios: Diseño / Transición / Operación del Servicio y Service Desk.

- Prácticas técnicas: modelos de servicios en la nube.

A su vez, cada una de estas prácticas se basa en cuatro dimensiones:

- Organización y personas, a través de una cultura y competencias corporativas.
- Información y tecnología, entendido como el conocimiento y la información existentes.
- Proveedores y socios, aquellas partes interesadas directamente involucradas en el servicio.
- Flujos de valor y procesos, creación de valor a través de la integración de todas las partes de la organización.

Los procesos por implantar son los siguientes:

- Prácticas generales de gestión
 - Gestión de la arquitectura
 - Mejora continua
 - Gestión de la seguridad de la información
 - Gestión del conocimiento
 - Medición y notificación
 - Gestión del cambio organizacional
 - Gestión de la cartera
 - Gestión de proyectos
 - Gestión de las relaciones
 - Gestión de riesgos
 - Gestión financiera de los servicios
 - Gestión de la estrategia
 - Gestión de Suministradores
 - Gestión de la fuerza de trabajo y del talento
- Prácticas de gestión de servicios
 - Gestión de la disponibilidad
 - Análisis de negocio
 - Gestión de la capacidad y el rendimiento
 - Control de cambios
 - Gestión de incidentes
 - Gestión de activos de TI

- Monitoreo y gestión de eventos
- Gestión de problemas
- Gestión de versiones
- Gestión de catálogos de servicios
- Gestión de la configuración del servicio
- Gestión de la continuidad del servicio
- Diseño del servicio
- Servicio de atención al cliente
- Gestión del nivel de servicio
- Gestión de peticiones de servicio
- Validación y prueba del servicio
- Prácticas de gestión técnica
 - Gestión de la implementación
 - Gestión de infraestructuras y plataformas
 - Desarrollo y gestión del software

Ventajas de la implantación de un nuevo modelo de gestión

El solo hecho de identificar cuáles son las principales fallas que presenta la prestación de un determinado servicio, hace posible el mejoramiento de todos los procesos. Por ello, aquí te contamos cuáles son los principales beneficios de la implementación de un modelo de gestión:

- Mayor control de los procesos.
- Estandarización de los procedimientos internos de la compañía.
- Flexibilidad y fácil adaptación a los cambios.
- Optimización de los recursos.
- Reducción de costes operacionales.
- Definir nuevos procesos alineados con los objetivos empresariales.

Pasos en implementación de la metodología ITIL

La implementación de la metodología ITIL en una empresa consta de los siguientes procesos:

- La creación del proyecto.
- La definición de la estructura de los servicios.
- La distinción de roles ITIL.

- El análisis de los procesos actuales de la empresa.
- La determinación de la estructura de procesos.
- La definición de las interfaces de procesos ITIL.
- La puntualización del control de procesos.
- El diseño de los procesos.
- La implantación de los sistemas de aplicaciones.
- La puesta en marcha de los procesos.
- La reeducación sobre el proceso.

Certificaciones ITIL

El programa ITIL cuenta con el aval y el respaldo de la ITIL Certification Management Board (ICMB). Este programa consta de varios exámenes que avalan los conocimientos en el área para obtener la certificación. En la actualidad, el Esquema de Certificación ITIL se divide en varios módulos en función del grado de profundidad que se desee aplicar. De esta manera, será mucho más fácil la selección de los procesos. Hoy en día, se encuentran disponibles cuatro tipos de certificación:

- Certificación ITIL Foundation
- Certificación ITIL Intermediate
- Certificación ITIL Expert
- Certificación ITIL Master

Bibliografía

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts, Tenth Edition*. Hoboken, New Jersey: John Wiley & Sons, Inc.

Stallings, W. (2018). *Operating Systems: Internals and Design Principles*, 9th Edition; Editorial: Pearson; ISBN-10: 0134670957; ISBN-13: 978-0134670959

Stallings, W. (2012). *Operating Systems, Internals and Design Principles, Seventh Edition*. Upper Saddle River; New Jersey: Pearson Education, Inc.

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems, Fourth Edition*. Upper Saddle River, New Jersey: Pearson Education, Inc.